

TST AI: ASISTENTE INTELIGENTE DE ESCRITORIO PARA PRODUCTIVIDAD Y CONFORT

Tamara Valeria Escobar Andrade y Santiago Rodríguez Bermeo

Facultad de Ingeniería, Programa Ingeniería de Sistemas

Corporación Universitaria del Huila – CORHUILA

Microcontroladores – Grupo I

Álvaro Hernán Alarcón López

Mayo 20, 2026

Tabla de Contenido

Introducción.....	5
Introduction.....	9
Objetivos	10
Objetivo General.....	10
Objetivos Específicos	10
Definición y Análisis del Problema.....	11
Marco Teórico.....	14
Microcontrolador ESP32.....	14
Sensor Ultrasónico (HC-SR04).....	14
Divisores de Tensión y Niveles Lógicos	15
Interfaz de Potencia.....	15
Comunicación Serial vs. Comunicación Paralela.....	16
Concepto y Utilidad de las Funciones en Programación	16
Sensor de Temperatura y Humedad DHT22.....	16
Transductor Acústico (Buzzer).....	17
Pantalla TFT LCD Táctil.....	17
Plataforma de Internet de las Cosas (IoT) ThingSpeak	18
Modulación por Ancho de Pulsos (PWM).....	18
Transistor de Efecto de Campo (MOSFET)	19
Agente Inteligente.....	19
Groq.....	20
Large Language Models (LLM).....	20
Prompt Engineering	20
Técnica Pomodoro.....	20
APIs REST.....	21
Flask	21
Diseño	23

Pines.....	23
Simulación	24
Alimentación	26
Desarrollo	29
Planeación y Bosquejo del Proyecto	29
Desarrollo del Hardware	30
Desarrollo e Implementación del Sistema.....	34
Integración IoT.....	39
Backend.....	42
Agente Inteligente.....	44
Frontend e interfaz del usuario	50
Simulación y Pruebas	54
Conclusiones.....	56
Referencias	57

Lista de Tablas

Tabla 1. Sensores y Actuadores	23
Tabla 2. Pantalla LCD TFT	24
Tabla 3. Organización modular del código del sistema.	35
Tabla 4. Resumen de la lógica del sistema.	38

Lista de Figuras

Figura 1. Módulo de desarrollo ESP32 DevKit. Fuente: [10]	14
Figura 2. Funcionamiento y temporización del sensor HC-SR04. Fuente: [11]	15
Figura 3. Circuito divisor de tensión pasivo. Fuente: [14].	15
Figura 4. Sensor de temperatura y humedad DHT22 (AM2302). Fuente: [19].	17
Figura 5. Alarma de zumbador piezoeléctrico de 12V. Fuente: [21].	17
Figura 6. Módulo de pantalla TFT LCD. Fuente: [23].	18
Figura 7. Principio de funcionamiento y estructura interna de un transistor MOSFET. Fuente: [27].	19
Figura 8. Conexiones de los dispositivos alimentados con 3.3 V	25
Figura 9. Conexión del HC-SR04 con el módulo regulador de voltaje	26
Figura 10. Conexión del ventilador con regulador de voltaje y mosfet	27
Figura 11. Conexión de la lámpara con fuente de 5V y mosfet	28
Figura 12. Diseños preliminares e ideas del funcionamiento	30
Figura 13. Compra de materiales	31
Figura 14. Conexión electrónica de sensores y actuadores provisionales	32
Figura 15. Proceso de impresión 3D de la carcasa	33
Figura 16. Circuito dentro de carcasa	33
Figura 17. Programación de la lógica en C++	34
Figura 18. Estructura del código ESP32	35
Figura 19. Implementación de la lógica del sistema en el hardware	37
Figura 20. Prueba de integración y funcionamiento del sistema	39
Figura 21. Configuración de los fields en Thingspeak	40
Figura 22. Código de comunicación de ESP32 y Thingspeak	41
Figura 23. Visualización de gráficas en Thingspeak	42
Figura 24. Programando el backend del sistema	43
Figura 25. Estructura modular del código del sistema	44
Figura 26. Creación de API-KEY del agente	45
Figura 27. Prompt para el agente inteligente	46
Figura 28. Funcionamiento de agente inteligente	48
Figura 29. Gráfica de solicitudes y consumo de tokens de la inteligencia artificial	49
Figura 30. Logs de funcionamiento de la API-KEY del agente inteligente	49
Figura 31. Interfaz web para ingreso y gestión de tareas	50
Figura 32. Propuesta de planificación generada por el agente inteligente	52

Figura 33. Temporizador Pomodoro durante la fase de trabajo	53
Figura 34. Recomendaciones inteligentes durante la fase de descanso.....	54
Figura 35. Prueba de funcionamiento final del sistema TST AI	55

Introducción

En la actualidad, los entornos laborales y académicos han experimentado una transformación digital acelerada, obligando a profesionales y estudiantes a pasar jornadas prolongadas frente a pantallas de computador. Esta transición ha traído consigo un incremento notorio en problemas de salud física y mental, destacando trastornos musculoesqueléticos debidos a malas posturas ergonómicas, fatiga visual por distancias inadecuadas de visualización y una disminución en la eficiencia cognitiva a causa de la falta de pausas activas organizadas. A pesar de la existencia de técnicas de gestión del tiempo de reconocida eficacia, como la técnica Pomodoro, los usuarios suelen experimentar dificultades para mantener la disciplina de forma autónoma sin un sistema que supervise y retroalimente sus hábitos en tiempo real.

Paralelamente, el avance tecnológico en las áreas de Internet de las Cosas (IoT) y la Inteligencia Artificial (IA) ha abierto nuevas fronteras para el desarrollo de sistemas embebidos de bajo costo y alto rendimiento. Los microcontroladores versátiles de última generación permiten la integración sinérgica de sensores ambientales, transductores ultrasónicos e interfaces gráficas locales, facilitando la captura y el procesamiento de variables del entorno físico con una latencia mínima. Al conectar este ecosistema de hardware con plataformas de telemetría en la nube y agentes inteligentes de software, es posible diseñar herramientas personalizadas que no solo monitorean el espacio de trabajo, sino que actúan activamente en beneficio del bienestar y confort del usuario.

El presente proyecto detalla el diseño, desarrollo e implementación de TST AI: Asistente Inteligente de Escritorio para Productividad y Confort. Este sistema integral actúa como un asistente ergonómico y organizador de actividades mediante la fusión de hardware embebido y software analítico. A través de un sensor ultrasónico, el dispositivo supervisa constantemente la distancia del usuario respecto al monitor, emitiendo alertas sonoras y visuales ante aproximaciones que comprometan la salud visual, al tiempo que automatiza la iluminación del escritorio según la presencia detectada. Asimismo, el sistema regula el confort térmico mediante un actuador de ventilación controlado por temperatura y gestiona de manera dinámica los ciclos de trabajo mediante un temporizador Pomodoro físico reflejado en una pantalla TFT LCD. Finalmente, la integración con plataformas en la nube asegura el almacenamiento histórico de datos, mientras que una interfaz web

conectada a un agente inteligente optimiza y prioriza la agenda diaria de tareas del usuario bajo criterios personalizados.

Introduction

In the contemporary era, accelerated digital transformation across academic and professional sectors has compelled individuals to endure prolonged hours in front of computer workstations. This shift has triggered a concerning rise in both physical and psychological health issues, most notably musculoskeletal disorders resulting from poor ergonomics, visual fatigue caused by inadequate viewing distances, and reduced cognitive efficiency due to a lack of structured breaks. Although well-established time-management strategies—such as the Pomodoro technique—exist to mitigate these effects, users frequently struggle to maintain the necessary discipline autonomously without a system capable of actively monitoring and reinforcing healthy habits in real time.

Concurrently, technological breakthroughs in the Internet of Things (IoT) and Artificial Intelligence (AI) have unlocked new frontiers for low-cost, high-performance embedded systems. Next-generation microcontrollers allow for the seamless integration of environmental sensors, ultrasonic transducers, and local graphical interfaces, enabling the capture and processing of physical variables with minimal latency. By bridging this hardware ecosystem with cloud-telemetry infrastructure and intelligent software agents, it is now entirely feasible to design personalized tools that move beyond mere environmental monitoring, actively intervening to enhance user well-being and ergonomic comfort.

This project presents the design, development, and implementation of TST AI: Smart Desktop Assistant for Productivity and Comfort. This comprehensive system acts as an ergonomic supervisor and task manager by merging embedded hardware with analytical software. Utilizing an ultrasonic sensor, the device continuously monitors the user's distance from the monitor, triggering both acoustic and visual alerts during hazardous proximity thresholds to safeguard visual health, while dynamically managing desk illumination based on presence detection. Furthermore, the system regulates thermal comfort via a temperature-controlled ventilation actuator and actively structures work cycles through a physical Pomodoro timer displayed on a TFT LCD screen. Finally, cloud integration ensures historical data logging, while a dedicated web interface powered by an intelligent agent optimizes and prioritizes the user's daily agenda based on custom parameters.

Objetivos

Objetivo General

Diseñar e implementar un sistema asistente de escritorio inteligente denominado TST AI, basado en el microcontrolador ESP32, que integre tecnologías de Internet de las Cosas (IoT) e Inteligencia Artificial para optimizar la productividad personal y mejorar las condiciones ergonómicas y de confort ambiental del usuario en entornos de trabajo prolongados.

Objetivos Específicos

- Desarrollar un sistema de supervisión ergonómica mediante sensores ultrasónicos que detecte la presencia del usuario y emita alertas preventivas ante distancias de proximidad críticas para la salud visual.
- Implementar mecanismos de control automático ambiental que regulen la temperatura y la iluminación del área de trabajo mediante la integración de sensores de precisión y actuadores de potencia.
- Integrar una interfaz basada en una pantalla TFT LCD para la visualización de métricas en tiempo real y la gestión de ciclos de trabajo bajo la metodología Pomodoro.
- Configurar una arquitectura de telemetría IoT en la plataforma ThingSpeak que permita el almacenamiento, monitoreo remoto y análisis histórico de las variables recolectadas por el sistema.
- Desarrollar un agente inteligente de gestión de tareas mediante una plataforma web que, a través de modelos de lenguaje avanzados, priorice la agenda del usuario y sugiera pausas activas personalizadas durante los periodos de descanso.

Definición y Análisis del Problema

Sistema inteligente de asistencia ergonómica y productividad para entornos de trabajo y estudio basados en computador. El sistema está orientado a usuarios que realizan actividades prolongadas frente al computador, como estudiantes, trabajadores remotos y personal de oficina, proporcionando asistencia ergonómica y apoyo inteligente para mejorar el confort y la productividad.

El uso prolongado del computador se ha convertido en una actividad cotidiana en contextos académicos y laborales. Sin embargo, esta práctica también se relaciona con diversos problemas de salud y rendimiento, especialmente cuando se realiza durante extensas jornadas sin pausas activas ni condiciones ergonómicas adecuadas. Diversos estudios evidencian que la exposición continua a pantallas puede generar síndrome visual informático, molestias musculoesqueléticas, fatiga mental y disminución de la productividad [1], [2]. En consecuencia, la problemática no radica únicamente en el tiempo de exposición, sino también en la ausencia de sistemas inteligentes capaces de asistir al usuario en el mantenimiento de hábitos saludables y condiciones adecuadas de trabajo.

Uno de los principales factores asociados al trabajo prolongado frente al computador es la fatiga visual. La revisión sistemática realizada por Sánchez et al. reporta que el síndrome visual informático presenta una alta prevalencia en trabajadores de oficina y personal que utiliza computadores de forma intensiva, manifestándose mediante síntomas como dolor de cabeza, visión borrosa, resequedad ocular y cansancio visual [1]. De manera complementaria, Hernández-Pavón et al. identificaron una prevalencia combinada del 71.33 % en trabajadores de oficina, estableciendo el tiempo prolongado de exposición al computador como uno de los principales factores de riesgo [2]. Estos hallazgos justifican la necesidad de implementar mecanismos de monitoreo y alerta que ayuden al usuario a mantener distancias adecuadas respecto al monitor y fomenten pausas preventivas.

A los problemas visuales se suman también alteraciones posturales derivadas de posiciones inadecuadas durante largas jornadas de trabajo o estudio. La permanencia continua frente al computador favorece tensiones en cuello, espalda y hombros, afectando el bienestar físico y el desempeño del usuario. Investigaciones relacionadas con escritorios ergonómicos inteligentes indican que la adaptación del entorno de trabajo puede mejorar significativamente la

comodidad y reducir la carga postural [3]. Asimismo, estudios sobre monitoreo postural en tiempo real demuestran que la detección automática de malas posturas contribuye a la prevención de riesgos ergonómicos y promueve hábitos más saludables [4].

Otro aspecto relevante es el confort térmico del entorno. La temperatura ambiental influye directamente sobre la concentración, el confort y el rendimiento laboral. Estudios sobre ambientes de trabajo señalan que temperaturas inadecuadas generan incomodidad física y disminuyen la capacidad de atención, afectando negativamente la productividad [5], [6]. Por esta razón, la incorporación de sistemas automáticos de ventilación representa una estrategia funcional para mantener condiciones ambientales más adecuadas durante periodos prolongados de trabajo.

Desde la perspectiva de la productividad, también existe una problemática relacionada con la sobrecarga cognitiva y la deficiente administración del tiempo. En escenarios académicos y laborales es frecuente que los usuarios acumulen tareas, reduzcan sus periodos de descanso y mantengan sesiones extensas de concentración, incrementando la fatiga mental y disminuyendo la eficiencia. La técnica Pomodoro ha demostrado ser una estrategia efectiva para mejorar la organización del tiempo mediante intervalos de trabajo y pausas programadas [7], [8]. Sin embargo, muchas personas no aplican esta metodología de manera constante debido a la falta de seguimiento o asistencia automatizada.

Adicionalmente, muchos entornos de trabajo y estudio carecen de herramientas de monitoreo remoto que permitan supervisar variables ambientales y analizar el comportamiento del usuario a lo largo del tiempo. La incorporación de plataformas IoT como ThingSpeak posibilita el almacenamiento y visualización de datos en la nube, facilitando el acceso remoto a información relacionada con temperatura, distancia y estados del sistema. Esto no solo mejora el seguimiento de las condiciones del entorno, sino que también permite realizar análisis históricos orientados a optimizar el confort y la productividad del usuario.

En este contexto, surge la necesidad de desarrollar un sistema inteligente de asistencia ergonómica y productividad para entornos de trabajo y estudio basados en computador. La propuesta integra sensores para monitoreo de presencia, distancia y temperatura, actuadores para iluminación y ventilación automática, y un agente de inteligencia artificial capaz de priorizar tareas, gestionar tiempos de

concentración y sugerir descansos mediante la técnica Pomodoro. De esta manera, el sistema busca contribuir al bienestar físico, reducir la fatiga derivada del uso prolongado del computador y optimizar la productividad del usuario mediante automatización e inteligencia artificial [3], [4], [6].

Marco Teórico

Microcontrolador ESP32

El ESP32 es un microcontrolador de alto rendimiento ampliamente utilizado en proyectos de electrónica y sistemas embebidos. Entre sus principales características se encuentran múltiples pines GPIO (General Purpose Input Output), conectividad WiFi y Bluetooth, y la incorporación de conversores analógico-digitales (ADC) [9]. Como se aprecia en su diagrama de pines (Figura 1), estas funciones permiten procesar señales tanto digitales como analógicas, facilitando el desarrollo de aplicaciones que requieren monitoreo y control en tiempo real.

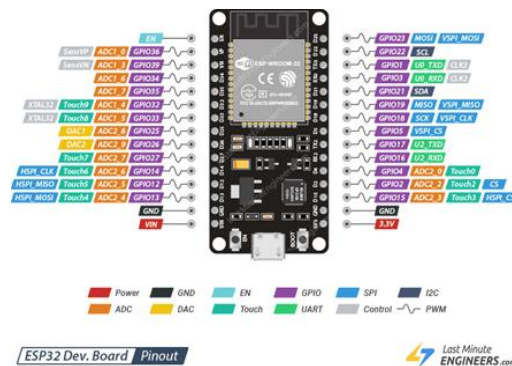


Figura 1. Módulo de desarrollo ESP32 DevKit. Fuente: [10]

Sensor Ultrasónico (HC-SR04)

El sensor HC-SR04 es un transductor que permite medir distancias mediante el uso de ondas de sonido ultrasónicas (frecuencias por encima del espectro audible humano, típicamente 40 kHz) [11].

- **Funcionamiento:** El sensor emite un pulso sónico (*Trigger*) y mide el tiempo que tarda la onda en rebotar contra un objeto y regresar al receptor (*Echo*) [11].



Figura 2. Funcionamiento y temporización del sensor HC-SR04. Fuente: [11]

- **Cálculo de distancia:** La distancia se calcula considerando la velocidad del sonido en el aire (aprox. 343 m/s) [11]. En MicroPython, se utiliza la fórmula $\text{distancia} = (\text{tiempo} / 2) / 29.1$ para obtener el valor en centímetros.

Divisores de Tensión y Niveles Lógicos

Dado que la ESP32 opera a un nivel lógico de 3.3V, es fundamental asegurar la compatibilidad con los sensores [12]. Mientras que el sensor PIR suele entregar 3.3V en su pin de salida, el HC-SR04 requiere una alimentación de 5V para un funcionamiento óptimo de sus transductores [13]. El manejo de señales entre diferentes niveles de voltaje es un concepto clave en la electrónica de microcontroladores para evitar daños permanentes en los pines de entrada [12].

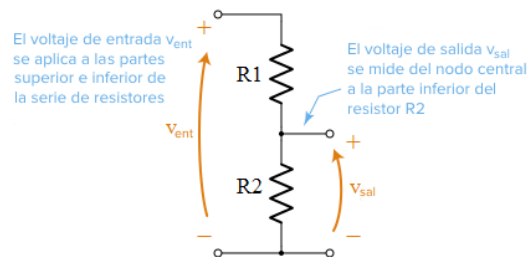


Figura 3. Circuito divisor de tensión pasivo. Fuente: [14].

Interfaz de Potencia

En el diseño de sistemas electrónicos de control, una interfaz de potencia es un circuito intermedio diseñado para acoplar la etapa lógica de control (baja potencia) con la etapa de actuación (alta potencia) [15]. Debido a que las salidas de un microcontrolador están limitadas en voltaje y corriente, la interfaz de potencia actúa como un puente que permite conmutar o regular grandes cargas eléctricas sin

comprometer la integridad del procesador, aislando las perturbaciones eléctricas mediante componentes semiconductores o electromecánicos [15].

Comunicación Serial vs. Comunicación Paralela

La transferencia de datos entre dispositivos digitales se clasifica principalmente según la disposición física de sus líneas de transmisión:

- **Comunicación Serial:** Consiste en el envío de datos bit a bit de manera secuencial a través de un único canal o conductor físico [16]. Aunque requiere una sincronización precisa, reduce drásticamente el cableado y es ideal para transmisiones a largas distancias.
- **Comunicación Paralela:** Permite la transmisión simultánea de múltiples bits (generalmente un byte o más) a través de un bus compuesto por múltiples líneas físicas en un mismo ciclo de reloj [16]. Presenta altas velocidades de transferencia a cortas distancias, pero es propensa al ruido por diafonía (*crosstalk*) y desalineación de bits (*skew*) en trayectos largos.

Concepto y Utilidad de las Funciones en Programación

En el desarrollo de software para sistemas embebidos, una función es un bloque de código estructurado, independiente y reutilizable diseñado para realizar una tarea específica dentro de un programa [17]. Su utilidad radica en la optimización del código fuente mediante la modularización, permitiendo subdividir problemas complejos en subrutinas más sencillas [17]. Esto facilita la legibilidad, evita la duplicación de instrucciones en memoria y simplifica los procesos de depuración y mantenimiento del sistema.

Sensor de Temperatura y Humedad DHT22

Para la adquisición de variables ambientales, se emplea el sensor digital DHT22 (también comercializado como AM2302). Este dispositivo integra un sensor capacitivo para determinar la humedad relativa y un termistor de coeficiente de temperatura negativo (NTC) para medir la temperatura del aire circundante [18]. A diferencia de variantes analógicas, el DHT22 realiza la conversión de la señal de manera interna y transmite los datos mediante un protocolo serie síncrono de un solo hilo (*single-wire*), lo que reduce el ruido en la línea de transmisión y optimiza el uso de los pines del microcontrolador [18].



Figura 4. Sensor de temperatura y humedad DHT22 (AM2302). Fuente: [19].

Transductor Acústico (Buzzer)

El zumbador o *buzzer* es un transductor electroacústico utilizado para la generación de alertas sonoras en sistemas embebidos. Su principio de operación se basa en el efecto piezoeléctrico inverso, donde la aplicación de una diferencia de potencial a través de un elemento cerámico piezoeléctrico induce una deformación mecánica cíclica [20]. Esta vibración desplaza el aire circundante, generando ondas sonoras a una frecuencia que puede ser controlada directamente mediante la señal eléctrica de excitación, lo que lo convierte en un periférico de salida eficiente y de bajo consumo de corriente para interfaces de usuario [20].



Figura 5. Alarma de zumbador piezoeléctrico de 12V. Fuente: [21].

Pantalla TFT LCD Táctil

La visualización de datos en el hardware se resuelve mediante una pantalla de cristal líquido con transistores de película delgada (TFT LCD, por sus siglas en

inglés). Esta tecnología implementa una matriz activa donde cada píxel es direccionado individualmente por un transistor dedicado, mejorando sustancialmente los tiempos de respuesta, el contraste y la saturación del color en comparación con las pantallas pasivas tradicionales [22]. Al acoplarse con un panel táctil (ya sea resistivo o capacitivo), el módulo se consolida como una Interfaz Hombre-Máquina (HMI) que permite tanto la salida de datos en tiempo real como la entrada de comandos de control por parte del operador [22].



Figura 6. Módulo de pantalla TFT LCD. Fuente: [23].

Plataforma de Internet de las Cosas (IoT) ThingSpeak

El almacenamiento, procesamiento y monitoreo remoto de las variables capturadas se gestiona a través de ThingSpeak. Esta plataforma de servicio en la nube está diseñada específicamente para aplicaciones de Internet de las Cosas (IoT), permitiendo la recepción de flujos de datos en tiempo real mediante API numéricas basadas en protocolos web estándar como HTTP y MQTT [24]. Una característica fundamental de este entorno es su integración nativa con herramientas de computación analítica como MATLAB, lo que facilita el procesamiento de datos históricos, el análisis estadístico y la visualización gráfica de las variables de manera remota [24].

Modulación por Ancho de Pulsos (PWM)

La modulación por ancho de pulsos (PWM) es una técnica de control empleada para regular la cantidad de potencia promedio que se entrega a una carga eléctrica a partir de una señal completamente digital [25]. El principio consiste en modificar el

ciclo de trabajo (*duty cycle*) de una señal cuadrada de frecuencia constante; el ciclo de trabajo define el porcentaje de tiempo que la señal permanece en un estado lógico alto respecto al período total de la onda [25]. Dado que el componente de control opera en conmutación (completamente encendido o apagado), la pérdida de energía en forma de calor se minimiza drásticamente.

Transistor de Efecto de Campo (MOSFET)

Para la etapa de potencia y el accionamiento de actuadores que demandan corrientes superiores a las soportadas por las salidas lógicas de un microcontrolador, se utiliza el MOSFET (Transistor de Efecto de Campo Metal-Óxido-Semiconductor) [26]. Este dispositivo semiconductor de tres terminales (Compuerta, Drenador y Fuente) opera por control de voltaje. Al aplicar un potencial adecuado en la compuerta (*Gate*), se genera un campo eléctrico que modula la conductividad de un canal semiconductor, permitiendo el flujo de corriente entre el drenador y la fuente [26]. Debido a su alta velocidad de conmutación y baja resistencia en estado de conducción $R_{DS(on)}$, es el componente ideal para acoplarse con señales PWM.

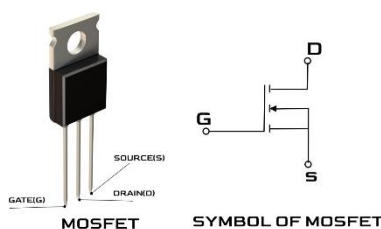


Figura 7. Principio de funcionamiento y estructura interna de un transistor MOSFET. Fuente: [27].

Agente Inteligente

Un agente inteligente es un sistema computacional capaz de percibir información del entorno, procesarla y tomar decisiones de manera autónoma con el objetivo de cumplir determinadas tareas o metas. Estos sistemas utilizan técnicas de inteligencia artificial para analizar datos, interpretar situaciones y generar respuestas adecuadas según el contexto [27]. Los agentes inteligentes son ampliamente utilizados en asistentes virtuales, automatización de procesos, sistemas de recomendación y aplicaciones de productividad. En el presente proyecto, el agente inteligente se encarga de analizar las tareas ingresadas por el usuario, priorizarlas y generar recomendaciones mediante la técnica Pomodoro.

Groq

Groq es una plataforma especializada en la ejecución de modelos de inteligencia artificial de gran escala mediante infraestructura optimizada para procesamiento de alto rendimiento. Esta plataforma permite acceder a modelos de lenguaje a través de APIs, facilitando su integración en aplicaciones web y sistemas inteligentes [28]. En este proyecto se utilizó Groq para ejecutar el modelo Llama 3 70B 8192, encargado de generar propuestas de organización y productividad basadas en las tareas del usuario.

Large Language Models (LLM)

Los Large Language Models (LLM) son modelos de inteligencia artificial entrenados con grandes volúmenes de texto con el fin de comprender y generar lenguaje natural. Estos modelos utilizan redes neuronales profundas y técnicas de aprendizaje automático para realizar tareas como generación de texto, razonamiento, traducción y respuesta a preguntas [29]. Dentro del proyecto, el modelo Llama 3 70B 8192 actúa como el núcleo del agente inteligente, permitiendo analizar tareas y generar recomendaciones personalizadas.

Prompt Engineering

El Prompt Engineering consiste en el diseño y estructuración de instrucciones enviadas a modelos de lenguaje con el propósito de obtener respuestas más precisas y controladas. Esta técnica permite definir reglas, formatos y objetivos específicos para mejorar el comportamiento de la inteligencia artificial [30]. En el desarrollo del proyecto, se diseñó un prompt estructurado que incluye restricciones de tiempo, formato JSON y criterios de priorización para garantizar que el agente inteligente genere respuestas válidas y coherentes.

Técnica Pomodoro

La técnica Pomodoro es un método de gestión del tiempo desarrollado por Francesco Cirillo, basado en intervalos de trabajo y descanso programados. Generalmente, esta técnica propone periodos de concentración de 25 minutos seguidos de pausas cortas, con el objetivo de mejorar la productividad y reducir la fatiga mental [31]. Diversos estudios han demostrado que la aplicación de pausas activas y sesiones de trabajo estructuradas contribuye al aumento de la concentración y al manejo eficiente del tiempo. En este proyecto, el agente

inteligente utiliza esta técnica para definir automáticamente tiempos de trabajo y descanso.

APIs REST

Las APIs REST (*Representational State Transfer*) son interfaces de comunicación utilizadas para el intercambio de información entre sistemas mediante protocolos HTTP. Estas APIs permiten enviar y recibir datos en formatos ligeros como JSON, facilitando la integración entre aplicaciones web, servicios en la nube y dispositivos IoT [32]. En el proyecto TST AI, las APIs REST fueron utilizadas para la comunicación entre el backend desarrollado en Flask, la página web y la ESP32.

Flask

Flask es un microframework de desarrollo web para Python que permite crear aplicaciones web y APIs de manera sencilla y modular [33]. Su estructura ligera facilita la implementación de rutas, servicios y comunicación entre diferentes componentes del sistema. En el presente proyecto, Flask fue utilizado para desarrollar el backend encargado de gestionar la lógica del sistema, la comunicación con el agente inteligente y la interacción entre la interfaz web y la ESP32.

Modelo Llama 3 70B 8192

Llama 3 70B 8192 es un modelo de lenguaje de gran escala desarrollado por Meta AI, diseñado para tareas de comprensión y generación de lenguaje natural. El término “70B” hace referencia a sus aproximadamente 70 mil millones de parámetros, lo que le permite realizar procesos avanzados de razonamiento, interpretación contextual y generación de respuestas coherentes [34].

Asimismo, el valor “8192” corresponde a la ventana de contexto del modelo, indicando la cantidad de tokens que puede procesar simultáneamente durante una interacción. Esto permite manejar instrucciones extensas, listas de tareas y reglas complejas dentro de una misma consulta [34].

En el presente proyecto, el modelo Llama 3 70B 8192 fue utilizado como núcleo del agente inteligente encargado de analizar las tareas ingresadas por el usuario, priorizarlas y generar propuestas Pomodoro junto con recomendaciones de descanso personalizadas.

Distribución del Equipo

Para asegurar el correcto desarrollo, la integración óptima de los subsistemas y el cumplimiento de los objetivos del proyecto TST AI, el equipo de trabajo se estructuró bajo una metodología de roles definidos, distribuyendo las responsabilidades técnicas y operativas de la siguiente manera:

1. Coordinador del Proyecto (Santiago Rodríguez Bermeo)

El coordinador asumió la dirección del desarrollo lógico, la integración de datos y la validación del ecosistema global. Sus funciones principales incluyeron:

- Integración IoT y Telemetría: Configuración, gestión y enlace de la API de la plataforma en la nube ThingSpeak para la recepción de flujos de datos históricos provenientes de la ESP32.
- Supervisión Lógica: Dirección del flujo de datos general del programa y control de la sincronización entre el hardware local y la plataforma web.
- Análisis de Resultados: Diseño y supervisión de las pruebas de simulación, recolección de métricas de rendimiento y validación del correcto funcionamiento de las alertas.

2. Socializadora del Proyecto (Tamara Valeria Escobar Andrade)

La socializadora se encargó de la fundamentación analítica, el diseño de la lógica algorítmica de alto nivel y la estructuración del documento técnico. Sus funciones principales incluyeron:

- Planteamiento e Investigación: Definición metodológica del análisis del problema, justificación ergonómica y formulación de los objetivos generales y específicos de la investigación.
- Lógica del Agente Inteligente: Diseño conceptual y desarrollo del algoritmo de inteligencia artificial encargado de la organización, priorización y optimización de las tareas diarias del usuario en la plataforma web.
- Comunicación Técnica: Redacción, control de calidad del informe de ingeniería y articulación de la transferencia de conocimiento del proyecto.

3. Asistente del Proyecto (Thomas Trujillo Cerquera)

El asistente asumió la responsabilidad directa sobre la arquitectura física, el cálculo de circuitos de instrumentación y la etapa de potencia del hardware. Sus funciones principales incluyeron:

- **Diseño y Ensamble de Hardware:** Selección, interconexión y calibración de los componentes físicos, incluyendo el microcontrolador ESP32, el sensor DHT22, el transductor ultrasónico HC-SR04 y la pantalla TFT LCD.
- **Desarrollo de la Interfaz de Potencia:** Diseño y montaje de la etapa de acoplamiento basada en transistores MOSFET de canal N, incluyendo el cálculo de las resistencias de pull-down y la integración de diodos flyback para la conmutación segura del ventilador y la lámpara.
- **Control de Modulación y Normativa:** Configuración de las señales de Modulación por Ancho de Pulsos (PWM) en el buzzer para la dosificación de frecuencias acústicas, y consolidación de la estructura bibliográfica del informe bajo la Normativa IEEE.

Diseño

Pines

Para realizar el diseño del circuito, se conectaron todos los pines de señal, así como las líneas de alimentación de 3.3 V y GND, utilizando una placa de expansión para la ESP32. En esta placa se configuró el jumper amarillo en 3.3 V, permitiendo que dicho voltaje estuviera disponible en los puntos VCC de la placa y facilitando de esta manera las conexiones de los diferentes componentes del circuito.

Los pines utilizados para el circuito fueron los siguientes:

Tabla 1. Sensores y Actuadores

Señal	Pin
Trigger	12
Echo	14

Lámpara	13
Buzzer	27
Motor	26
DHT22	33

Tabla 2. Pantalla LCD TFT

Señal	Pin
MOSI	23
SCK	18
CS	21
DC	22
RST	19

Uno de los componentes utilizados fue el sensor DHT22, encargado de medir la temperatura del entorno. Dependiendo del valor detectado, la ESP32 enviaba una señal de control al ventilador para encenderlo cuando la temperatura superaba el umbral establecido y si no superaba el umbral, el ventilador permanecía apagado.

Simulación

Asimismo, se realizó la conexión de la pantalla TFT LCD de 2.8" y del dispositivo Buzzer directamente a la placa del circuito, conectando sus respectivos pines de alimentación y señal. Ambos dispositivos fueron alimentados con 3.3 V provenientes de la placa de expansión.

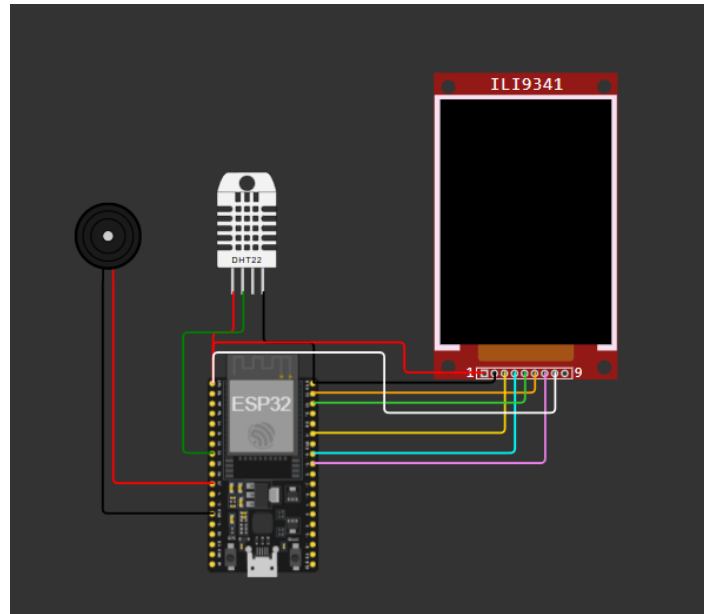


Figura 8. Conexiones de los dispositivos alimentados con 3.3 V

Por otro lado, el sensor ultrasónico y algunos actuadores, como el ventilador y la lámpara, requerían una alimentación externa. Para ello, se utilizó un cargador de 9 V y 3 A, el cual alimentaba tanto la placa de expansión como otros componentes del circuito. Sin embargo, debido a que el sensor ultrasónico y el ventilador operan a 5 V, fue necesario utilizar un módulo regulador de voltaje que redujera los 9 V de entrada a 5 V, evitando así daños en estos dispositivos.

El sensor ultrasónico presentaba una consideración importante: el pin ECHO entrega señales de 5 V, mientras que los pines GPIO de la ESP32 trabajan a 3.3 V. Para evitar posibles daños en la tarjeta, se implementó un divisor de voltaje utilizando tres resistencias de 1 kΩ, con el objetivo de reducir la señal de 5 V a aproximadamente 3.3 V.

El voltaje de salida del divisor de tensión se calculó mediante la siguiente ecuación:

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2}$$

$$V_{out} = 5V \cdot \frac{2k\Omega}{1k\Omega + 2k\Omega}$$

$$V_{out} = 3.333V$$

De esta forma, la señal enviada al pin GPIO de la ESP32 se mantuvo dentro de los niveles seguros de operación.

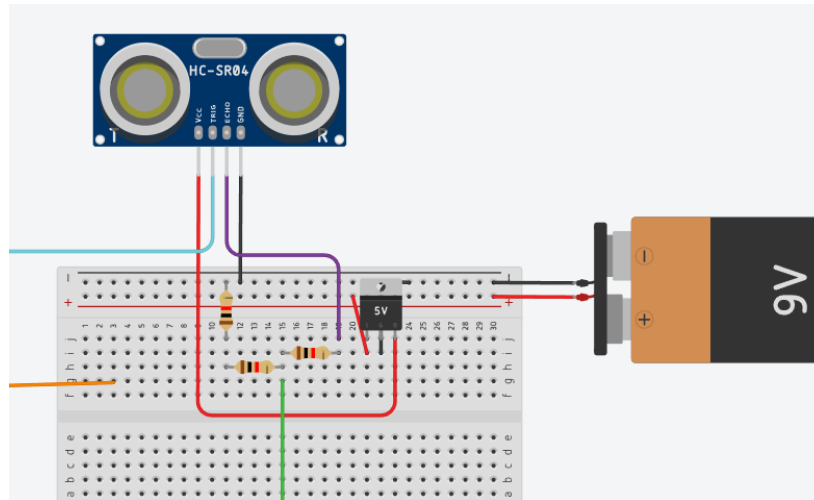


Figura 9. Conexión del HC-SR04 con el módulo regulador de voltaje

Alimentación

Para la lámpara se utilizó una fuente independiente de 5 V y 1 A, la cual proporcionó la alimentación necesaria para su correcto funcionamiento.

Además, se emplearon dos MOSFET IRLZ44N para controlar el encendido y apagado tanto de la lámpara como del ventilador. En ambos casos se realizaron conexiones similares.

En el pin Drain de cada MOSFET se conectó el ánodo de un diodo 1N4007, con el propósito de asegurar que la corriente circulara en una sola dirección y proteger el circuito frente a posibles picos de voltaje o ruido eléctrico generados por las cargas inductivas. El cátodo del diodo se conectó al VCC del actuador y a la línea de alimentación de 5 V, mientras que el ánodo se conectó tanto al pin Drain del MOSFET como al terminal GND del actuador.

En el pin Gate del MOSFET se conectó una resistencia en serie con la señal proveniente de la ESP32, utilizada para reducir ruido y proteger el pin GPIO.

Adicionalmente, se conectó otra resistencia entre el Gate y GND funcionando como resistencia pull-down, manteniendo el Gate en estado bajo 0o 0V cuando no existiera señal de control. Finalmente, el pin Source de cada MOSFET se conectó directamente a GND.

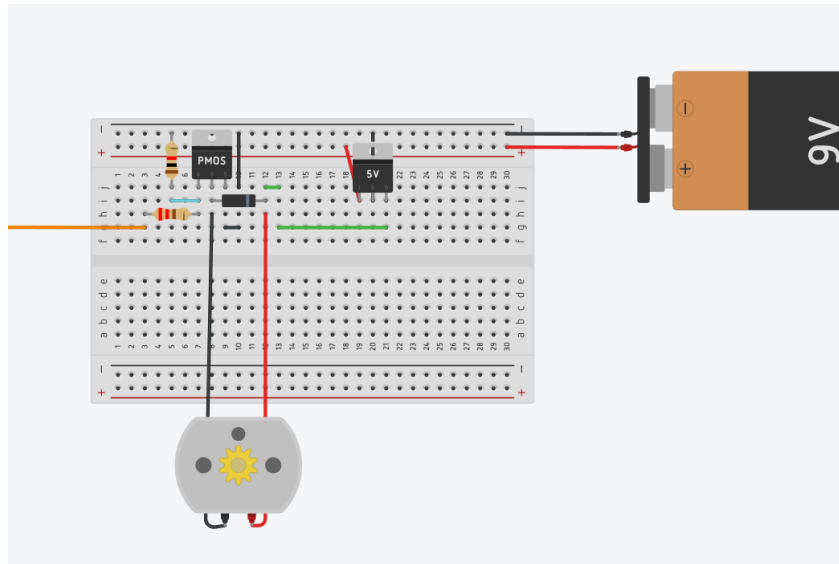


Figura 10. Conexión del ventilador con regulador de voltaje y mosfet

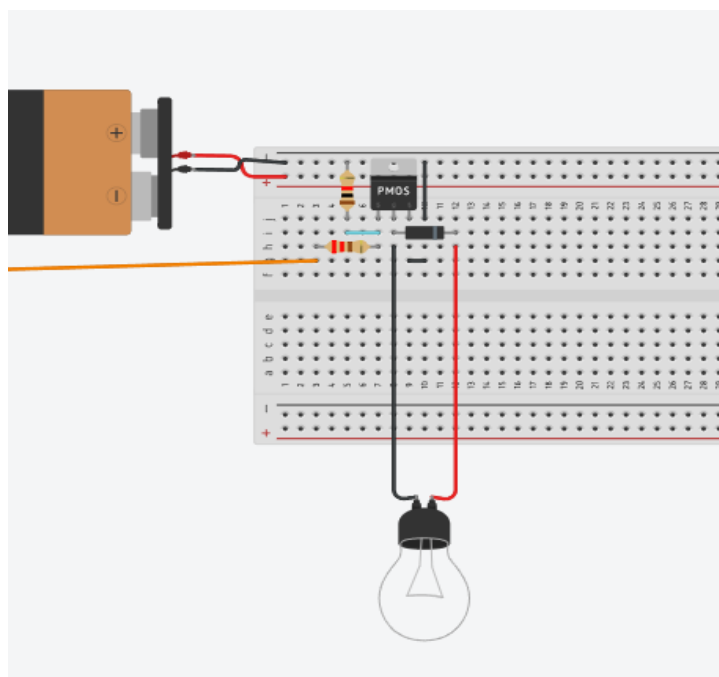


Figura 11. Conexión de la lámpara con fuente de 5V y mosfet

Desarrollo

Planeación y Bosquejo del Proyecto

Durante la etapa de planeación se realizaron diferentes bosquejos e ideas preliminares para definir tanto la funcionalidad como el diseño físico del proyecto. Después de analizar varias propuestas, se seleccionó el desarrollo de un asistente inteligente de escritorio enfocado en mejorar el confort y la productividad durante el trabajo en computador.

En esta etapa también se definieron las principales funciones del sistema, incluyendo la visualización de información en pantalla, alertas de proximidad, monitoreo de temperatura, temporizador Pomodoro y mensajes relacionados con el

estado del entorno. Asimismo, se seleccionaron los sensores y actuadores más adecuados para cumplir con los objetivos planteados.

Adicionalmente, se propuso un diseño interactivo con temática de gato para la carcasa del dispositivo, buscando no solo proteger el circuito electrónico, sino también brindar una apariencia más amigable y atractiva para el usuario.

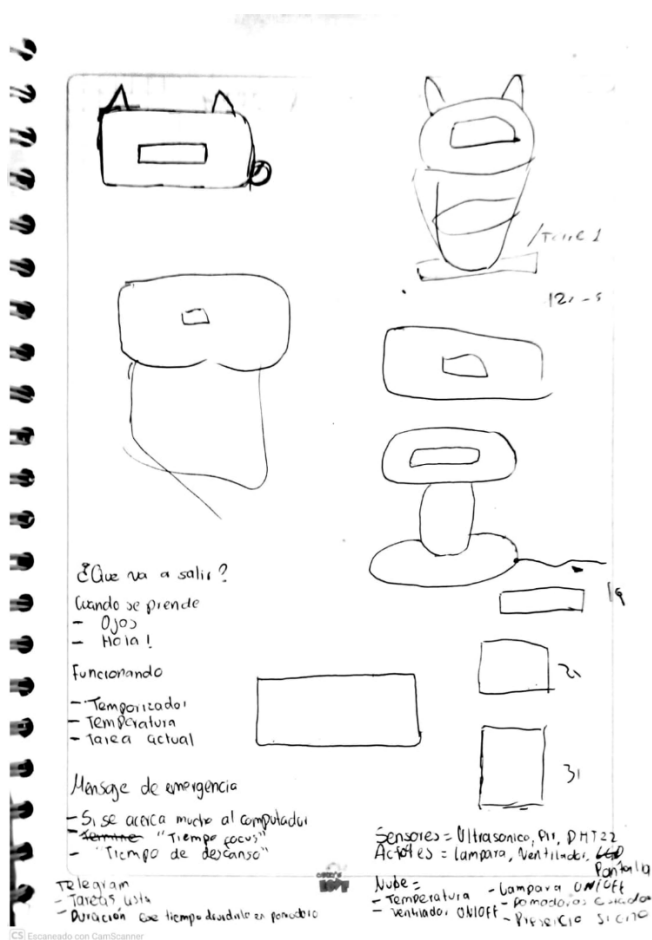


Figura 12. Diseños preliminares e ideas del funcionamiento

Desarrollo del Hardware

Una vez definidas las funcionalidades principales del proyecto, se realizó la adquisición de los materiales y componentes electrónicos necesarios para la implementación del sistema, como se observa en la Figura 6. Entre los elementos utilizados se encuentran la ESP32, sensor ultrasónico, baquela, sensor DHT22, MOSFET, ventilador, buzzer y pantalla TFT.



Figura 13. Compra de materiales

Posteriormente, se llevaron a cabo las conexiones electrónicas provisionales en la protoboard de los sensores y actuadores con el objetivo de verificar el correcto funcionamiento del sistema antes de su integración final. En esta etapa se realizaron pruebas de lectura de sensores, activación de actuadores y comunicación entre los diferentes módulos, tal como se muestra en la Figura 7.

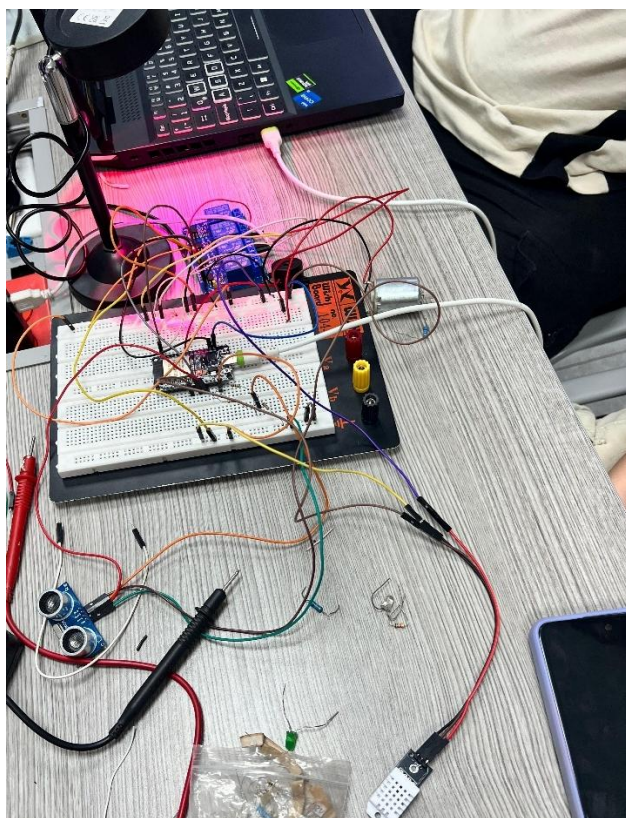


Figura 14. Conexión electrónica de sensores y actuadores provisionales

Además, se tuvo en cuenta la correcta distribución de alimentación del sistema, considerando los diferentes niveles de voltaje requeridos por la ESP32, sensores y actuadores, garantizando estabilidad y protección de los componentes electrónicos mediante el uso adecuado de módulos y conexiones de potencia.

Durante las pruebas iniciales se realizaron ajustes en las conexiones y calibraciones de los sensores con el fin de mejorar la estabilidad y precisión del sistema, especialmente en la medición de distancia y control de actuadores.

De manera paralela, se desarrolló el diseño de la carcasa mediante modelado 3D, teniendo en cuenta la distribución interna de los componentes, la protección del circuito y la estética general del dispositivo. El diseño preliminar de la carcasa y su tapa fue preparado en el software Cura, como se evidencia en la Figura 8.

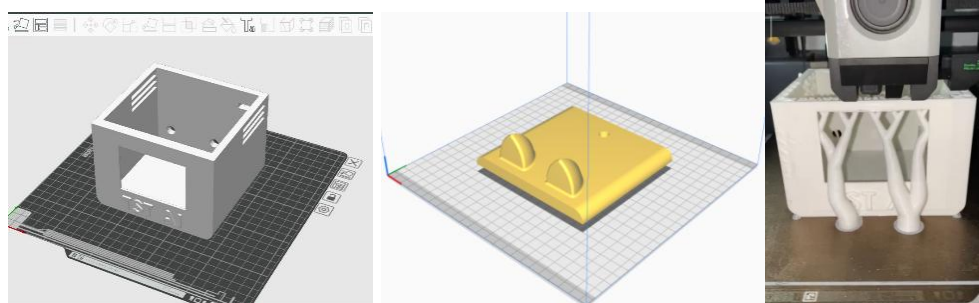


Figura 15. Proceso de impresión 3D de la carcasa

Posteriormente, se realizó el proceso de soldadura de las conexiones electrónicas con el fin de garantizar una integración más estable y segura de los componentes del sistema. Una vez finalizado este proceso, el circuito fue ensamblado e instalado dentro de la carcasa impresa en 3D, permitiendo una mejor organización del cableado, protección de los componentes y una presentación más estética del prototipo final, como se observa en la Figura 9.

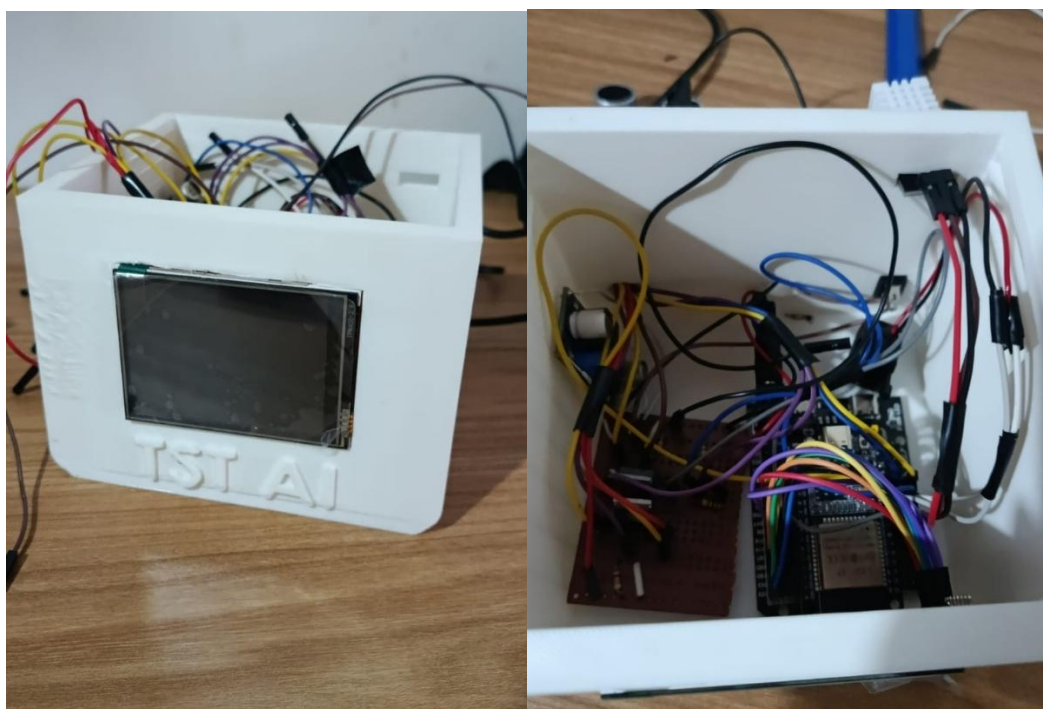


Figura 16. Circuito dentro de carcasa

Desarrollo e Implementación del Sistema

Para el desarrollo de la programación del sistema se utilizó el lenguaje C++, debido a la compatibilidad y disponibilidad de librerías necesarias para el manejo de la pantalla TFT utilizada en el proyecto. Durante esta etapa se implementó la lógica principal del sistema, incluyendo la lectura de sensores, control de actuadores, visualización de datos y comunicación.

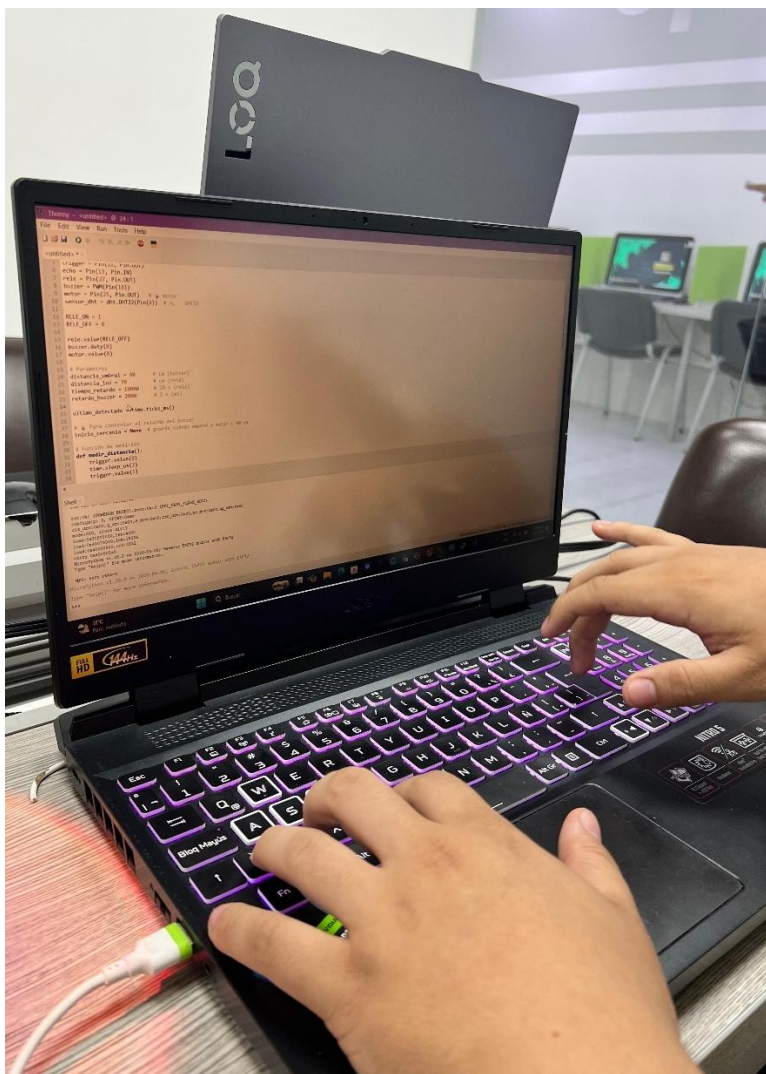


Figura 17. Programación de la lógica en C++

El código del sistema fue organizado de manera modular con el objetivo de facilitar su lectura, mantenimiento e integración con los diferentes componentes del

proyecto. Para ello, se dividió en archivos independientes encargados de funciones específicas, como sensores, actuadores, pantalla, conexión WiFi y comunicación con Flask.

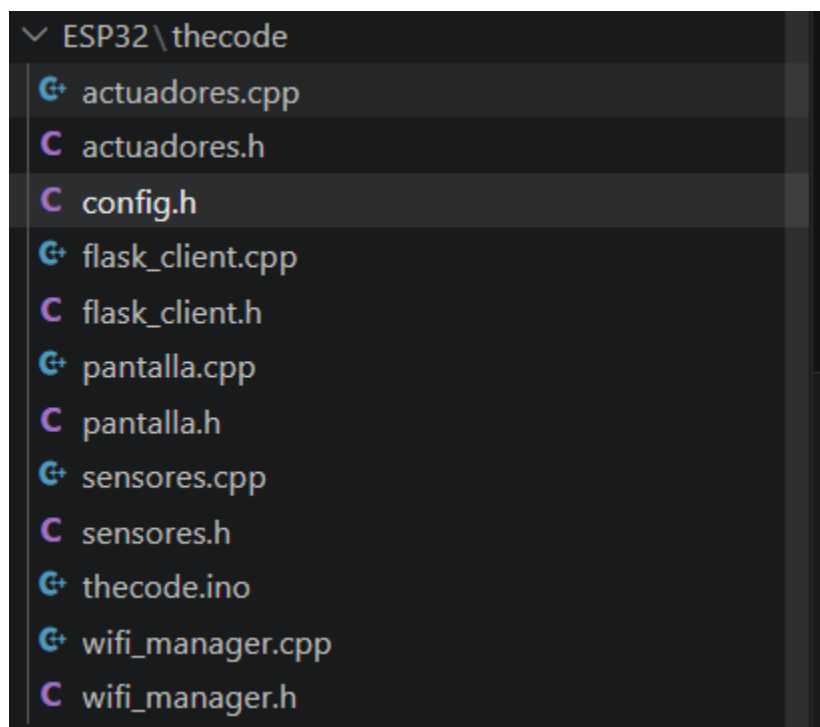


Figura 18. Estructura del código ESP32

En esta estructura se utilizaron archivos con extensión .h y .cpp. Los archivos .h (header) contienen las declaraciones de funciones, variables y configuraciones que pueden ser utilizadas en diferentes partes del programa, mientras que los archivos .cpp contienen la implementación o lógica de dichas funciones. Esta separación permitió mejorar la organización y modularidad del sistema, facilitando el desarrollo y depuración del código.

Tabla 3. Organización modular del código del sistema.

Archivo	Función principal
thecode.ino	Archivo principal del programa. Ejecuta setup() y loop().
config.h	Define pines, constantes y parámetros generales.
sensores.cpp / sensores.h	Lectura del sensor ultrasónico y DHT22.
actuadores.cpp / actuadores.h	Control de lámpara, ventilador y buzzer.
pantalla.cpp / pantalla.h	Manejo de la pantalla TFT y visualización de datos.
wifi_manager.cpp / wifi_manager.h	Conexión de la ESP32 a la red WiFi.
flask_client.cpp / flask_client.h	Comunicación con el backend Flask.

Posteriormente, se integró toda la lógica desarrollada con el hardware del proyecto, permitiendo la interacción entre sensores, actuadores y pantalla TFT. En esta etapa se configuró la visualización de información del entorno, como temperatura,

distancia detectada y estado de los actuadores, mostrando los datos en tiempo real sobre la pantalla del dispositivo.

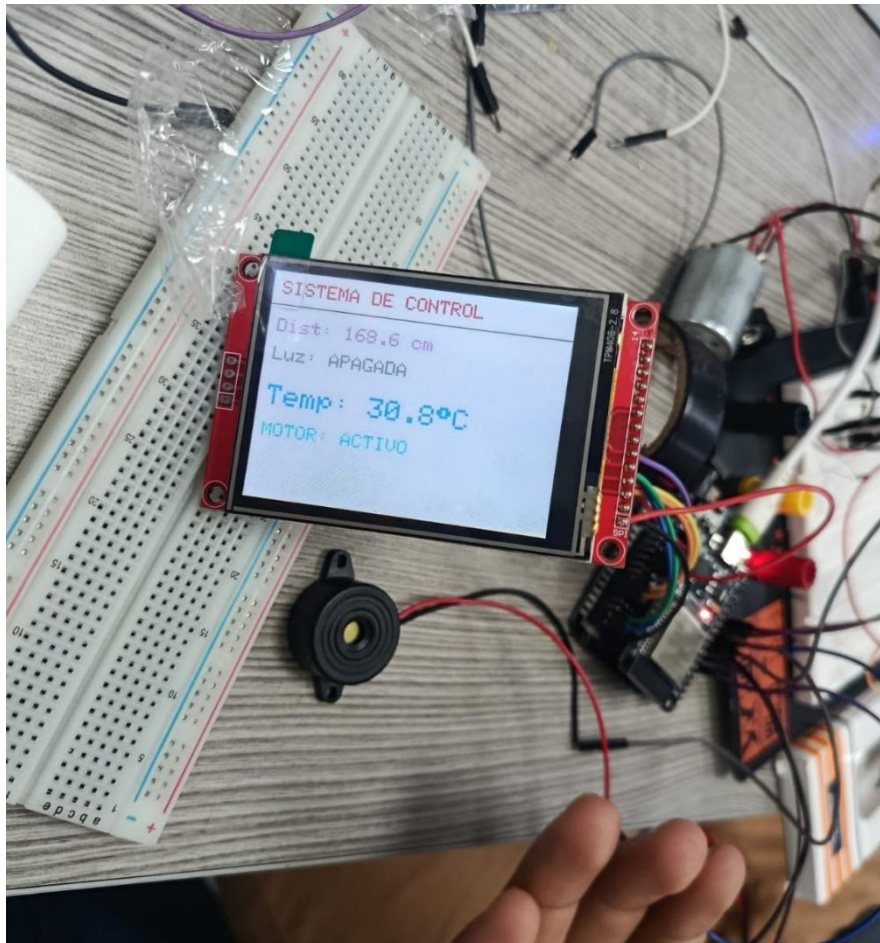


Figura 19. Implementación de la lógica del sistema en el hardware

La lógica general del sistema se basa en la detección de presencia del usuario mediante el sensor ultrasónico. Cuando el usuario se encuentra dentro del rango establecido, el sistema activa automáticamente la lámpara y el ventilador. Adicionalmente, si la distancia detectada es menor a 40 cm, el buzzer genera una alerta sonora con el objetivo de advertir una posible cercanía excesiva a la pantalla del computador.

Asimismo, el sensor DHT22 monitorea continuamente la temperatura del entorno. Cuando la temperatura supera los 30 °C, el ventilador se activa automáticamente para mejorar las condiciones de confort del usuario.

Tabla 4. Resumen de la lógica del sistema.

Condición detectada	Acción realizada
Usuario dentro 0-70 cm	Encendido de lámpara y ventilador
Distancia menor a 40 cm	Activación de buzzer
Temperatura mayor a 30 °C	Activación del ventilador
Usuario fuera de 0-70 cm	Apagado del ventilador
Usuario fuera del rango por 10 s	Apagado de lámpara

Finalmente, se realizaron pruebas de integración y funcionamiento para verificar la correcta operación de todos los módulos del sistema, incluyendo sensores, actuadores, visualización en pantalla

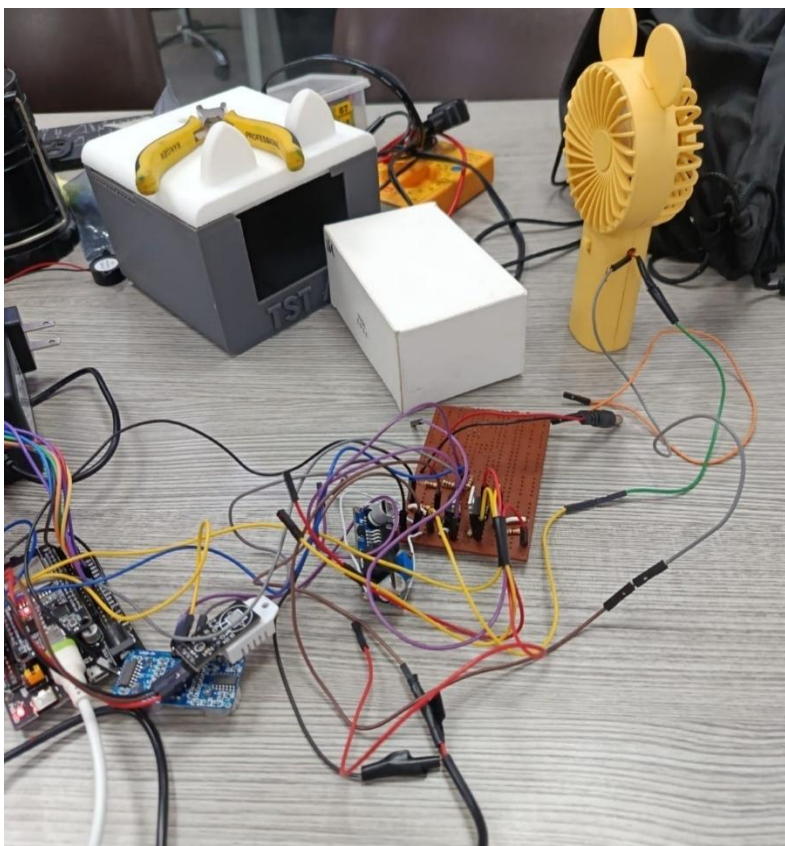


Figura 20. Prueba de integración y funcionamiento del sistema

Integración IoT

Con el objetivo de implementar capacidades de monitoreo remoto y visualización de datos en la nube, se integró la plataforma ThingSpeak dentro del sistema. Esta integración permitió almacenar y visualizar en tiempo real las variables medidas por los sensores conectados a la ESP32, facilitando el seguimiento del estado del entorno y del funcionamiento del asistente inteligente.

Inicialmente, se realizó la configuración de los diferentes campos (*fields*) dentro de la plataforma ThingSpeak, asignando cada variable del sistema a un canal específico para su posterior visualización y análisis. Entre las variables registradas se encuentran la temperatura, distancia detectada y estado de los actuadores.

Private View
Public View
Channel Settings
Sharing
API Keys
Data Import / Export

Channel Settings

Percentage Complete 30%

Channel ID 3366469

Name Datos Asistente

Description

Field 1 temperatura ☒

Field 2 distancia ☒

Field 3 presencia ☒

Field 4 ventilador ☒

Field 5 lampara ☒

Field 6 buzzer ☒

Field 7 ☐

Field 8 ☐

Metadata

Tags

Help

Channels store all the data that a ThingSpeak application collects. Each channel includes eight fields that can hold any type of data, plus three fields for location data and one for status data. Once you collect data in a channel, you can use ThingSpeak apps to analyze and visualize it.

Channel Settings

- Percentage complete:** Calculated based on data entered into the various fields of a channel. Enter the name, description, location, URL, video, and tags to complete your channel.
- Channel Name:** Enter a unique name for the ThingSpeak channel.
- Description:** Enter a description of the ThingSpeak channel.
- Fields:** Check the box to enable the field, and enter a field name. Each ThingSpeak channel can have up to 8 fields.
- Metadata:** Enter information about channel data, including JSON, XML, or CSV data.
- Tags:** Enter keywords that identify the channel. Separate tags with commas.
- Link to External Site:** If you have a website that contains information about your ThingSpeak channel, specify the URL.
- Show Channel Location:**
 - Latitude:** Specify the latitude position in decimal degrees. For example, the latitude of the city of London is 51.5072.
 - Longitude:** Specify the longitude position in decimal degrees. For example, the longitude of the city of London is -0.1275.
 - Elevation:** Specify the elevation position meters. For example, the elevation of the city of London is 35.052.
- Video URL:** If you have a YouTube™ or Vimeo® video that displays your channel information, specify the full path of the video URL.
- Link to GitHub:** If you store your ThingSpeak code on GitHub®, specify the GitHub repository URL.

Using the Channel

Figura 21. Configuración de los fields en Thingspeak

Posteriormente, se desarrolló la lógica de comunicación entre la ESP32 y la plataforma ThingSpeak mediante conexión WiFi y solicitudes HTTP. Esta comunicación permitió el envío periódico de los datos capturados por los sensores hacia la nube.


```
C config.h  thingspeak.py
nuevocodigo > backend > backend > services > thingspeak.py > enviar_a_thingspeak
1  import requests
2  import time
3
4  WRITE_API_KEY = "5TEXFSD670HA1IER"
5
6  URL = "https://api.thingspeak.com/update"
7
8  # Control de frecuencia
9  ultimo_envio = 0
10
11 INTERVALO_THINGSPEAK = 15 # segundos
12
13
14 def enviar_a_thingspeak(datos):
15     global ultimo_envio
16
17     print("[THINGSPEAK] Intentando enviar...")
18
19     ahora = time.time()
20
21     if ahora - ultimo_envio < INTERVALO_THINGSPEAK:
22         print("[THINGSPEAK] Esperando intervalo de 15s...")
23         return
24
25     ultimo_envio = ahora
26
27     payload = {
28         "api_key": WRITE_API_KEY,
29         "field1": datos.get("temperatura", 0),
30         "field2": datos.get("distancia", 0),
31         "field3": datos.get("presencia", 0),
32         "field4": datos.get("actuadores", {}).get("ventilador", 0),
33         "field5": datos.get("actuadores", {}).get("lampara", 0),
34         "field6": datos.get("actuadores", {}).get("buzzer", 0),
35     }
36
37     try:
38         response = requests.post(URL, data=payload)
39         print(f"[THINGSPEAK] Código: {response.status_code}")
40         print(f"[THINGSPEAK] Respuesta: {response.text}")
41
42     except Exception as e:
43         print(f"[THINGSPEAK] Error: {e}")
```

Figura 22. Código de comunicación de ESP32 y Thingspeak

Finalmente, los datos enviados fueron representados mediante gráficas que se actualizan cada 10 segundos dentro de la plataforma, permitiendo visualizar el comportamiento de las variables monitoreadas y facilitando el análisis del entorno de trabajo. En estas gráficas es posible observar la variación de la temperatura y la distancia a lo largo del tiempo, así como los valores registrados en cada intervalo de medición. Adicionalmente, se incorporaron indicadores para representar el estado de activación de los actuadores, utilizando los mismos colores mostrados en

la pantalla TFT con el fin de mantener coherencia visual entre la interfaz física y la plataforma IoT.

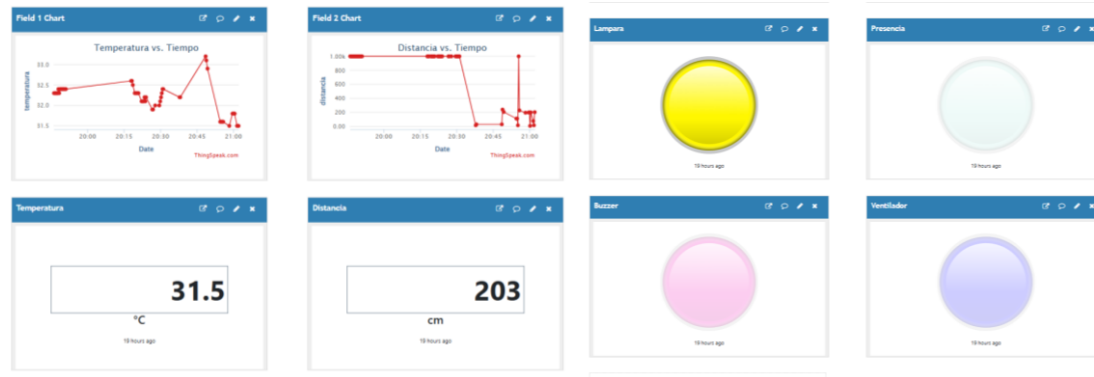


Figura 23. Visualización de gráficas en Thingspeak

Backend

Con el objetivo de complementar las funciones de automatización del sistema, se desarrolló una página web que permite la interacción entre el usuario, la ESP32 y el agente inteligente. A través de esta interfaz, el usuario puede ingresar tareas, visualizar información del entorno y consultar el estado general del sistema en tiempo real.

El desarrollo del software se realizó utilizando Flask como framework principal para el backend, permitiendo gestionar la comunicación entre la interfaz web, la ESP32 y los servicios de inteligencia artificial. Durante esta etapa se implementaron las diferentes rutas, servicios y funciones necesarias para el correcto funcionamiento del sistema.

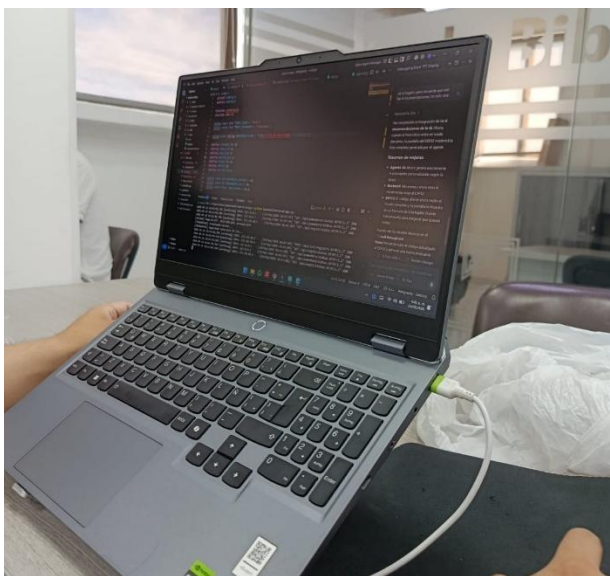


Figura 24. Programando el backend del sistema

La estructura del código fue organizada de manera modular con el fin de facilitar el mantenimiento, escalabilidad y comprensión del proyecto. Para ello, el sistema se dividió en carpetas encargadas de funciones específicas, como manejo de rutas, servicios, archivos estáticos y plantillas HTML.

Dentro de la carpeta services se implementaron los módulos principales relacionados con el agente inteligente, la lógica Pomodoro, el monitoreo del estado del sistema y la integración con ThingSpeak. Asimismo, las carpetas templates y static fueron utilizadas para el desarrollo de la interfaz web y los recursos visuales de la aplicación.

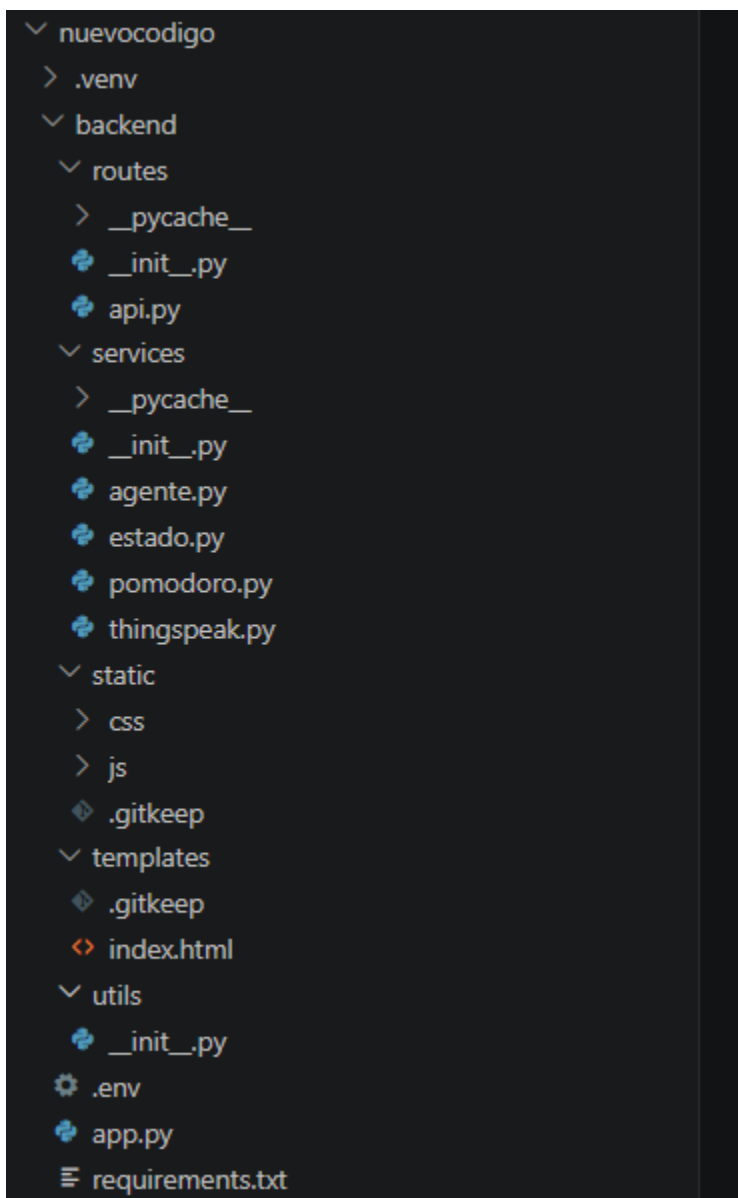


Figura 25. Estructura modular del código del sistema

Agente Inteligente

Para la integración del agente inteligente se utilizó una API basada en modelos de inteligencia artificial de Groq. Durante esta etapa se realizó la creación y

configuración de la API-KEY necesaria para establecer comunicación entre el sistema y el modelo de IA.

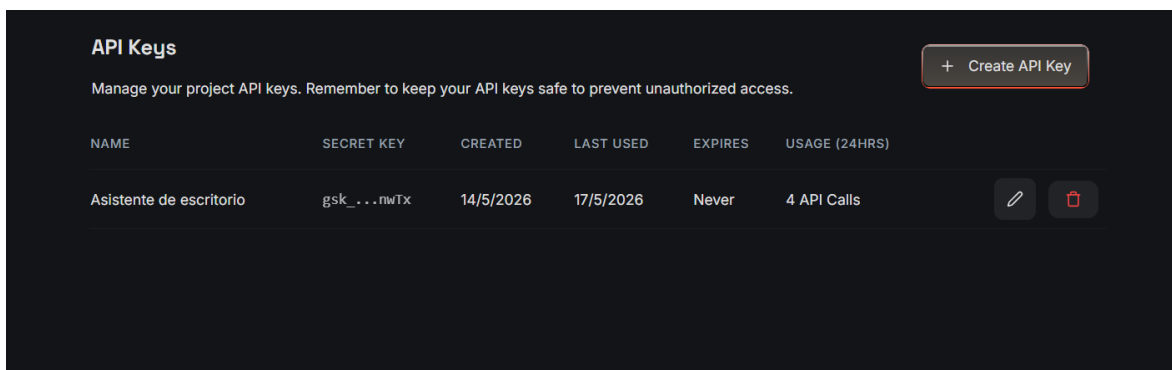


Figura 26. Creación de API-KEY del agente

Antes de profundizar en el agente se realizó el prompt que será entregado al LLM. El agente inteligente utiliza un prompt estructurado diseñado para orientar al modelo de inteligencia artificial en la generación de propuestas Pomodoro personalizadas. Dentro del prompt se incluyen las tareas ingresadas por el usuario junto con una serie de reglas obligatorias relacionadas con prioridad, dificultad, concentración, tiempos de trabajo y descanso. Además, se especifica el formato exacto de respuesta en JSON para garantizar que el sistema pueda procesar correctamente la información generada por el modelo. El prompt también solicita recomendaciones de descanso personalizadas y la priorización automática de tareas, permitiendo que el agente genere respuestas coherentes, organizadas y adaptadas al contexto del usuario.

```
def _construir_prompt(tareas: list) -> str:
    """
    Construye el prompt para el LLM.
    Prompt estructurado y explícito para maximizar la calidad del JSON de salida.
    """
    tareas_json = json.dumps(tareas, ensure_ascii=False, indent=2)

    return f"""Eres un agente experto en productividad y técnica Pomodoro.

    Analiza las siguientes tareas y genera una planificación Pomodoro óptima.

    TAREAS:
    {tareas_json}

    REGLAS OBLIGATORIAS:
    - Más dificultad + más concentración = mayor prioridad.
    - Si la concentración de la tarea más prioritaria es baja (1 o 2), elige en cambio una tarea más simple.
    - minutos_trabajo debe ser un número entero entre 15 y 45.
    - minutos_descanso debe ser un número entero entre 5 y 15.
    - actividades_descanso: exactamente 4 actividades breves. CRÍTICO: Deben ser ALTAMENTE PERSONALIZADAS e inspiradas en la temática de la tarea.
    NUNCA sugieras actividades que duren exactamente todo el tiempo de descanso (ej. si el descanso es 5 min, no sugieras caminar 5 min, sugiere cosas rápidas de 1 o 2 min). ¡Sé creativo y no repitas!

    - tipo_descanso debe ser exactamente uno de: activo, relajacion, social, creativo.
    - tareas_priorizadas: lista COMPLETA de tareas ordenadas de mayor a menor prioridad, con razón en español.

    Responde ÚNICAMENTE con un objeto JSON válido. No uses markdown, no añadas texto antes ni después del JSON.

    Formato exacto requerido:
    {
      "tareas_priorizadas": [
        {
          "tarea_id": "id de la tarea",
          "tarea_nombre": "nombre de la tarea",
          "prioridad": 1,
          "razon": "motivo breve en español"
        }
      ],
      "tarea_id": "id de la tarea elegida",
      "tarea_nombre": "nombre de la tarea elegida",
      "minutos_trabajo": 25,
      "minutos_descanso": 5,
      "actividades_descanso": [
        "actividad saludable 1",
        "actividad saludable 2",
        "actividad saludable 3",
        "actividad saludable 4"
      ],
      "tipo_descanso": "relajacion",
      "mensaje": "explicación breve en español de por qué elegiste esta tarea y estos tiempos"
    }
    """
```

Figura 27. Prompt para el agente inteligente

El código del agente inteligente se encarga de recibir una lista de tareas ingresadas por el usuario y generar una propuesta de organización basada en prioridad, tiempo de trabajo y descanso. Para esto, utiliza la API de Groq con el modelo llama3-70b-8192, el cual analiza las tareas según su dificultad y nivel de concentración requerido.

Primero, el programa carga la API-KEY desde el archivo .env, evitando escribirla directamente dentro del código. Luego intenta inicializar el cliente de Groq; si la API no está disponible o ocurre algún error, el sistema utiliza una lógica local de respaldo llamada propuesta_local(), lo que permite que el agente siga funcionando aunque no haya conexión con la IA.

La función principal del agente es generar_propuesta_pomodoro(tareas), la cual llama a run_agente(). Esta función construye un prompt con las tareas recibidas y solicita al modelo que devuelva una respuesta en formato JSON. La respuesta debe incluir la lista de tareas priorizadas, la tarea recomendada, los minutos de trabajo, los minutos de descanso, actividades sugeridas y un mensaje explicativo.

Además, el código incluye funciones de validación para asegurar que la respuesta de la IA sea correcta. Por ejemplo, verifica que el tiempo de trabajo esté entre 15 y 45 minutos, que el descanso esté entre 5 y 15 minutos, que la tarea seleccionada exista y que las actividades de descanso sean válidas. Si algún dato es incorrecto, el sistema lo corrige automáticamente.

En caso de que Groq falle, el sistema usa una priorización local basada en la suma de dificultad y concentración. La tarea con mayor puntaje se considera de mayor prioridad. También se personalizan las recomendaciones de descanso según el tipo de tarea, por ejemplo si es mental, física o relacionada con computador.

En resumen, este código permite que la página web genere una planificación inteligente tipo Pomodoro, organizando las tareas del usuario y proponiendo descansos adecuados para mejorar la productividad y concentración.

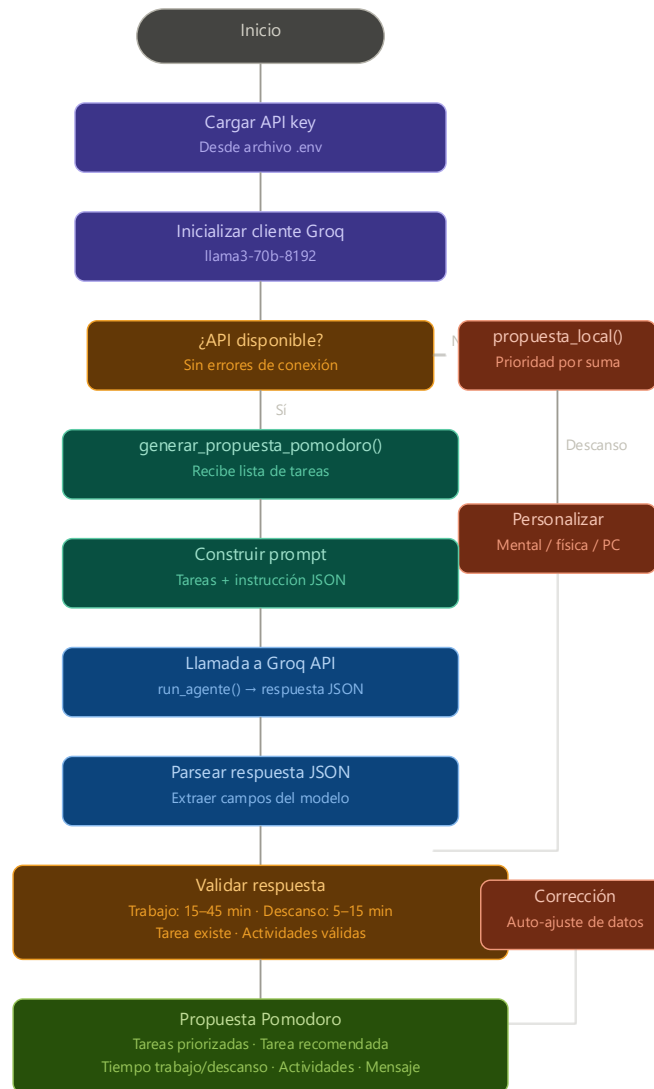


Figura 28. Funcionamiento de agente inteligente

Adicionalmente, se realizaron pruebas de funcionamiento y monitoreo del consumo de solicitudes (requests) y tokens utilizados por la API de inteligencia artificial. Estas métricas permitieron verificar la correcta comunicación con el modelo y evaluar el comportamiento del sistema durante su ejecución, tal como se evidencia a continuación:

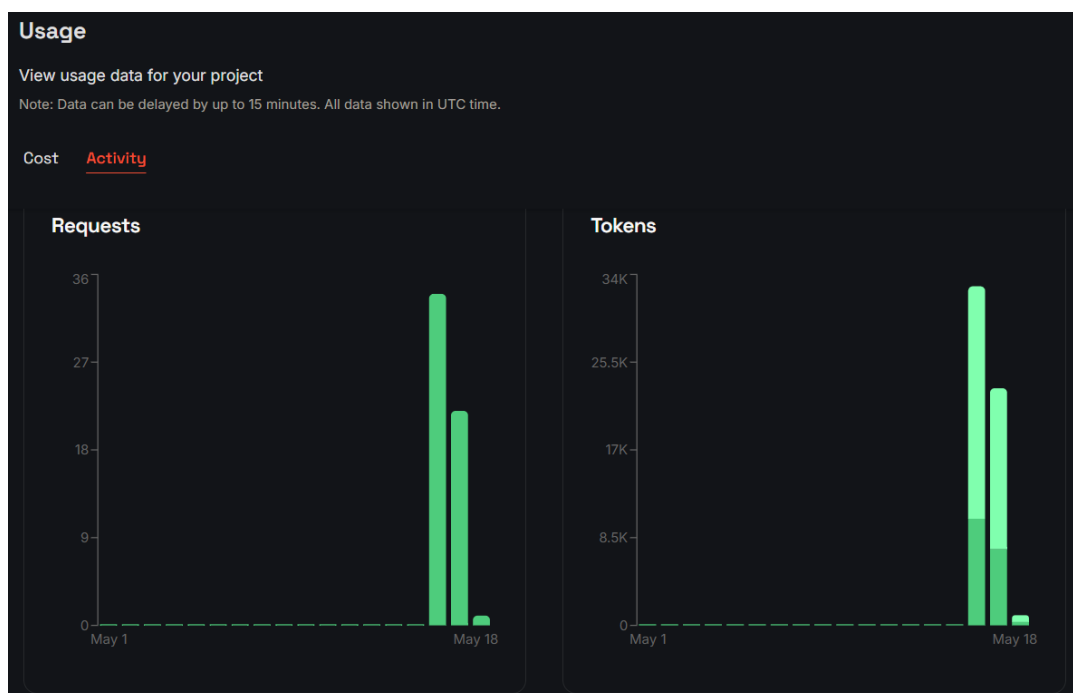


Figura 29. Gráfica de solicitudes y consumo de tokens de la inteligencia artificial

Finalmente, se verificó el funcionamiento general de la API mediante el análisis de los registros (*logs*) generados durante la comunicación entre el backend y el agente inteligente, permitiendo identificar el estado de las solicitudes y posibles eventos del sistema

Logs Show Errors Only Download

REQUEST TIME	MODEL	API KEY	CODE	TTFT	LATENCY	INPUT TOKENS	OUTPUT TOKENS	AUDIO SECONDS	REQUEST ID	ERROR
17/5/2026, 8:54:59 p. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.184	1.072	661	323	-	req_0...txw0	-
17/5/2026, 6:18:17 p. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.091	0.729	655	275	-	req_0...7csf	-
17/5/2026, 6:16:38 p. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.211	1.262	886	545	-	req_0...h5df	-
17/5/2026, 4:06:20 p. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.090	1.284	886	566	-	req_0...ahud	-
17/5/2026, 3:18:23 p. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.101	0.770	700	334	-	req_0...07qk	-
17/5/2026, 3:17:17 p. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.114	0.873	749	366	-	req_0...pg0p	-
17/5/2026, 12:33:44 p. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.117	0.720	657	254	-	req_0...8cs1	-
17/5/2026, 12:31:19 p. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.210	0.858	659	269	-	req_0...kcct	-
17/5/2026, 10:35:32 a. m.	llama-3.3-70b-versatile	Asistente de escritorio	200	0.326	0.956	655	241	-	req_0...6m3l	-

Figura 30. Logs de funcionamiento de la API-KEY del agente inteligente

Frontend e interfaz del usuario

Con el objetivo de complementar las funciones de automatización del sistema, se desarrolló una página web que permite la interacción entre el usuario, la ESP32 y el agente inteligente. A través de esta interfaz, el usuario puede ingresar tareas, visualizar información del entorno y consultar el estado general del sistema en tiempo real.



Figura 31. Interfaz web para ingreso y gestión de tareas.

El desarrollo del software se realizó utilizando Flask como framework principal para el backend, permitiendo gestionar la comunicación entre la interfaz web, la ESP32 y los servicios de inteligencia artificial. Durante esta etapa se implementaron las

diferentes rutas, servicios y funciones necesarias para el correcto funcionamiento del sistema.

La estructura del código fue organizada de manera modular con el fin de facilitar el mantenimiento, escalabilidad y comprensión del proyecto. Para ello, el sistema se dividió en carpetas encargadas de funciones específicas, como manejo de rutas, servicios, archivos estáticos y plantillas HTML. Dentro de la carpeta services se implementaron los módulos principales relacionados con el agente inteligente, la lógica Pomodoro, el monitoreo del estado del sistema y la integración con ThingSpeak.

Para la integración del agente inteligente se utilizó una API basada en modelos de inteligencia artificial de Groq, específicamente el modelo **Llama 3 70B 8192**, el cual permitió procesar y analizar las tareas ingresadas por el usuario. El agente desarrollado tiene la capacidad de organizar tareas según prioridad y dificultad, definir tiempos de trabajo y descanso mediante la técnica Pomodoro y generar recomendaciones personalizadas orientadas a mejorar la productividad y concentración del usuario.

Una vez procesadas las tareas, el sistema genera automáticamente una propuesta de planificación que incluye la tarea recomendada, nivel de prioridad, tiempo de trabajo y descanso, así como mensajes explicativos relacionados con la decisión tomada por el agente.

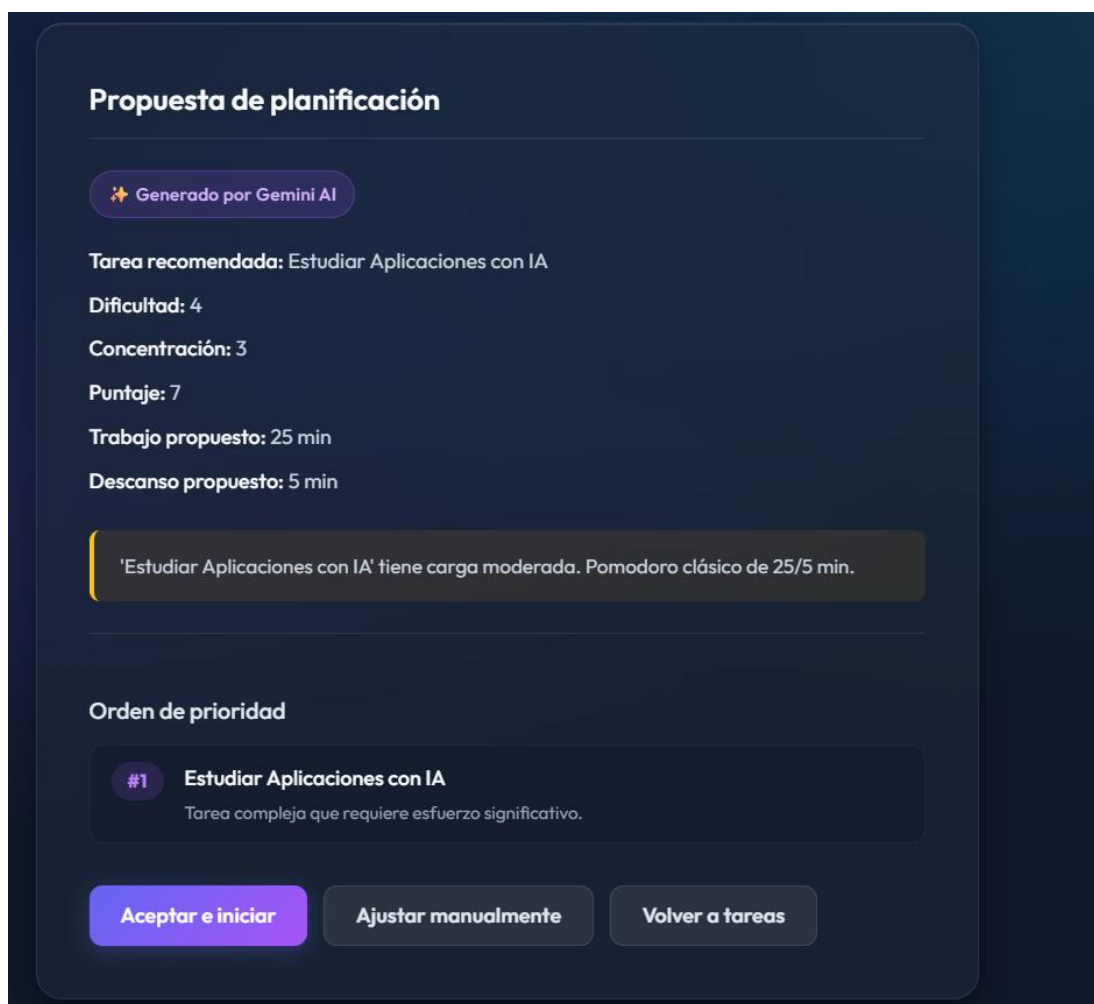


Figura 32. Propuesta de planificación generada por el agente inteligente.

Posteriormente, el usuario puede iniciar la ejecución del temporizador Pomodoro directamente desde la interfaz web. Durante la fase de trabajo, el sistema muestra el tiempo restante y la tarea actualmente seleccionada, permitiendo al usuario mantener seguimiento continuo de la actividad en ejecución.



Figura 33. Temporizador Pomodoro durante la fase de trabajo

Asimismo, durante la fase de descanso el sistema muestra recomendaciones personalizadas de pausas activas y actividades sugeridas por el agente inteligente. Estas recomendaciones fueron diseñadas para ayudar a reducir la fatiga visual y mental generada por el trabajo prolongado frente al computador

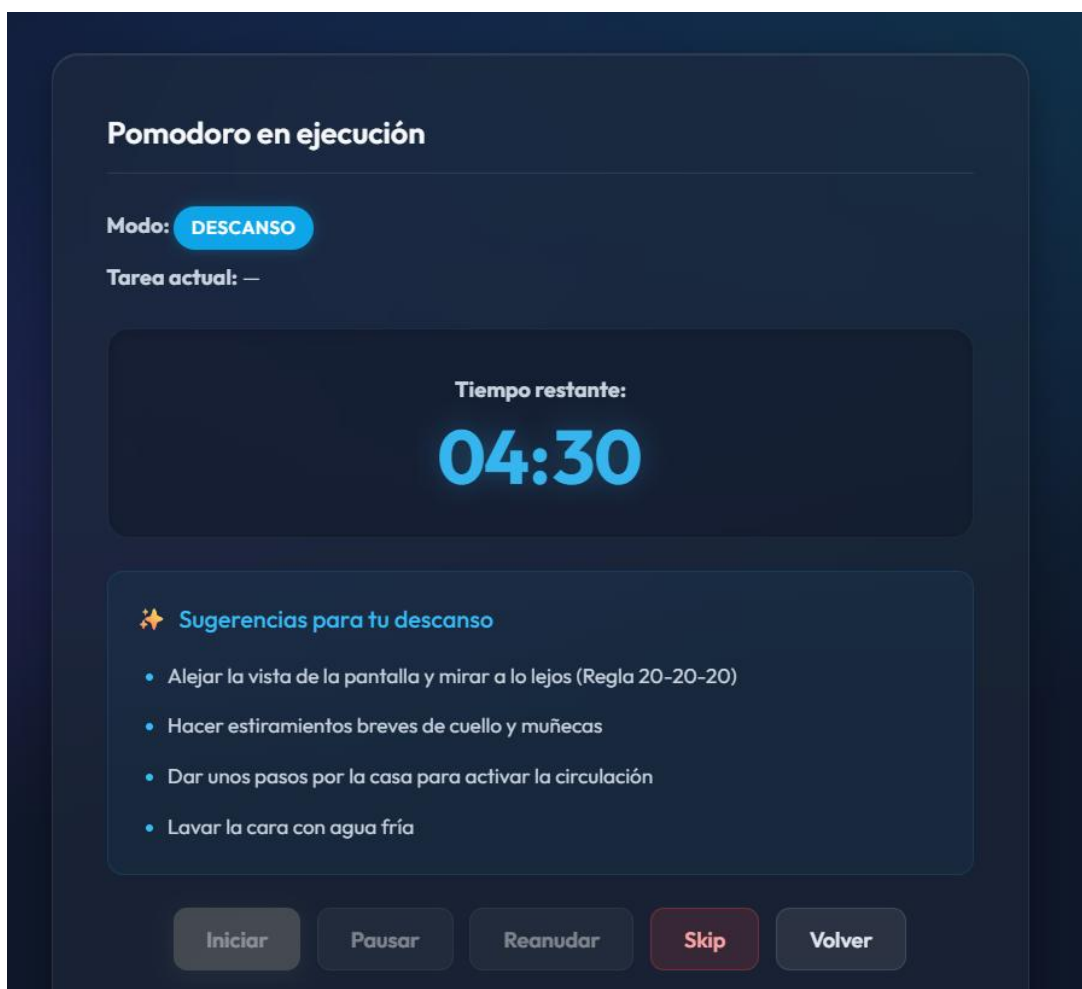


Figura 34. Recomendaciones inteligentes durante la fase de descanso.

Por último, el temporizador y las actividades recomendadas son sincronizados tanto en la interfaz web como en la pantalla TFT conectada a la ESP32, permitiendo mantener una visualización simultánea del estado del sistema en ambos entornos.

Simulación y Pruebas

Finalmente, se realizaron pruebas de integración y validación con el objetivo de verificar el correcto desempeño de todos los módulos implementados dentro del sistema. Durante estas pruebas se evaluó la interacción entre sensores, actuadores, pantalla TFT, plataforma ThingSpeak, página web y agente inteligente, comprobando la adecuada comunicación entre cada uno de los componentes del proyecto.

Asimismo, se verificó la correcta ejecución de las diferentes lógicas implementadas, incluyendo la detección de presencia, el control automático de lámpara y ventilador, las alertas de proximidad, el monitoreo de temperatura, la visualización de datos y la organización de tareas mediante inteligencia artificial.



Figura 35. Prueba de funcionamiento final del sistema TST AI

Adicionalmente, se adjunta un enlace en Google Drive correspondiente al video demostrativo del proyecto, en el cual se evidencia la integración y operación general de todos los componentes tanto de hardware como de software:
<https://drive.google.com/file/d/1GQcZTR0kRA0mAZTCB3ue1ugdXq3iJFn3/view>

Conclusiones

Se logró diseñar e implementar con éxito el asistente de escritorio inteligente TST AI, demostrando que la integración de hardware embebido (ESP32) con plataformas de Internet de las Cosas (IoT) y agentes de inteligencia artificial constituye una solución tecnológica viable, eficiente y de bajo costo para mitigar los riesgos ergonómicos y optimizar la productividad en entornos de trabajo digital prolongado.

La incorporación del sensor ultrasónico HC-SR04 acoplado a un sistema de alertas visuales en la pantalla TFT LCD y acústicas mediante modulación por ancho de pulsos (PWM) en el buzzer demostró una alta efectividad. El uso de PWM permitió calibrar diferentes frecuencias y tonos en el zumbador.

El control automatizado de los actuadores de potencia mediante etapas independientes basadas en transistores MOSFET garantizó una gestión eficiente tanto del confort térmico como de la iluminación ambiental. Este enfoque por conmutación permitió accionar el ventilador periférico de forma autónoma según las lecturas del sensor DHT22, así como encender o apagar la lámpara de escritorio según los estados de presencia o ausencia determinados por el sensor ultrasónico, optimizando el consumo energético del sistema.

La integración bidireccional entre el microcontrolador y los servicios de software (el almacenamiento de telemetría en la nube a través de la API de ThingSpeak y el procesamiento logístico de tareas mediante la interfaz web asistida por IA) consolida un ecosistema IoT completo. Esto permite un seguimiento de las variables ambientales y ergonómicas, al tiempo que delega la carga cognitiva de la planificación diaria en un agente inteligente.

La correcta adaptación de niveles lógicos entre los periféricos de 5V (como el transductor ultrasónico) y las entradas de 3.3V de la ESP32 mediante divisores de tensión pasivos validó la importancia del diseño riguroso de circuitos en sistemas de instrumentación electrónica, asegurando la estabilidad operativa del hardware y evitando daños permanentes en el microcontrolador.

Referencias

- [1] D. C. Sánchez et al., "Síndrome Visual Informático en trabajadores que usan computadores: revisión sistemática," *Revista de Salud Ocupacional*, 2022.
- [2] J. L. Hernández-Pavón et al., "Síndrome visual del computador en trabajadores de oficina," *Ergonomía, Investigación y Desarrollo*, vol. 7, no. 1, pp. 92–105, 2025, doi: 10.29393/EID7-7SVJA20007.
- [3] "Modelo de un escritorio ergonómico e inteligente para instructor y estudiante en un ambiente de formación académica moderno," *Revistas SENA*, 2022.
- [4] M. H. et al., "Real-Time Postures Detection and Monitoring at Workplaces Using Artificial Intelligence," *IEEE Xplore*, 2023.
- [5] Instituto de Salud Pública de Chile, "Confort térmico en ambientes laborales," Nota Técnica N° 47, 2017.
- [6] "The effects of temperature on work performance in the typical office environment: A meta-analysis of the current evidence," *ScienceDirect*, 2025.
- [7] F. Cirillo, *The Pomodoro Technique*. New York, NY, USA: Currency, 2018.
- [8] "Boosting Academic Writing Productivity with the Pomodoro Technique," 2023.
- [9] Deepsea Developments, "ESP32 Chip Explained: Features and Advantages," Deepsea Dev Blog, 2023. [En línea]. Disponible en: <https://www.deepseadev.com/en/blog/esp32-chipexplained-and-advantages/>
- [10] MCI Electronics, "Introducción a ESP32," MCI Electronics Blog, 05-jul-2024. [En línea]. Disponible en: <https://cursos.mcielectronics.cl/2024/07/05/introduccion-a-esp32/>
- [11] "11. Sensor Ultrasonico," Documento de clase sobre sensores de distancia, CORHUILA.
- [12] Random Nerd Tutorials, "ESP32 Pinout Reference: Which GPIO pins should you use?," 2023. [En línea]. Disponible en: <https://randomnerdtutorials.com/esp32-pinout-reference-gpios/>
- [13] Components101, "HC-SR04 Ultrasonic Sensor Pinout, Features & Datasheet," 2020. [En línea]. Disponible en: <https://components101.com/sensors/hc-sr04-ultrasonic-sensor>

- [14] W. McAllister, "Diagrama esquemático de un circuito divisor de voltaje," Khan Academy en Español, 2025. [Ilustración en línea]. Disponible en: <https://es.khanacademy.org/a/ee-voltage-divider>
- [15] M. H. Rashid, Electrónica de potencia: circuitos, dispositivos y aplicaciones, 4a ed. México: Pearson Educación, 2014.
- [16] W. Stallings, Data and Computer Communications, 10a ed. Upper Saddle River, NJ: Pearson, 2014.
- [17] R. E. Bryant y D. R. O'Hallaron, Computer Systems: A Programmer's Perspective, 3a ed. Boston, MA: Pearson, 2016.
- [18] Adafruit Industries, "DHT22 temperature-humidity sensor datasheet," Adafruit Learning System, 2026. [En línea]. Disponible en: <https://cdn-shop.adafruit.com/datasheets/Digital+humidity+and+temperature+sensor+AM2302.pdf>
- [19] Dualtrónica, "Sensor de temperatura y humedad DHT22 AM2302," Dualtrónica Tienda en Línea, 2026. [Fotografía en línea]. Disponible en: <https://dualtronica.com/inicio/415-sensor-de-temperatura-y-humedad-dht22-am2302.html?srsId=AfmBOopd102zIzodHkCkGLVGz8pw4nEa5wIAB8fyQ0uVJNrS1IDKDu81>
- [20] Cuviko, "Qué es un buzzer o zumbador piezoeléctrico y cómo funciona," Cuviko Blog, 2024. [En línea]. Disponible en: <https://www.cuviko.com/es/blog/que-es-un-buzzer-o-zumbador/>
- [21] FlyRobo, "12V Piezo Electric Buzzer Alarm," FlyRobo E-commerce Store, 2026. [Fotografía en línea]. Disponible en: <https://www.flyrobo.in/12v-piezo-electric-buzzer-alarm?srsId=AfmBOopof0hdddg9J5o6EagTjRyNVgtjrDTag96mEISr8D8oybtRkFiD>
- [22] Display Future, "What is TFT LCD Display?," Display Future Technical Articles, 2025. [En línea]. Disponible en: <https://www.displayfuture.com/en/what-is-tft-lcd-display/>
- [23] MercadoLibre Colombia, "Pantalla TFT LCD," catálogo de productos electrónicos, 2026. [Fotografía en línea]. Disponible en: <https://listado.mercadolibre.com.co/pantalla-tft-lcd>

- [24] MathWorks, "ThingSpeak IoT Platform with MATLAB Analytics," MathWorks Documentation, 2026. [En línea]. Disponible en: <https://thingspeak.com/>
- [25] SparkFun Electronics, "Pulse Width Modulation (PWM)," SparkFun Tutorials, 2023. [En línea]. Disponible en: <https://learn.sparkfun.com/tutorials/pulse-width-modulation/all>
- [26] Electronics Tutorials, "NMOS and PMOS MOSFET Transistor Basics," Discrete Semiconductors Reference, 2024. [En línea]. Disponible en: https://www.electronics-tutorials.ws/transistor/tran_7.html
- [27] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.
- [28] Groq Inc., "Groq Documentation," Groq, 2026. [Online]. Available: <https://groq.com/>
- [29] T. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [30] OpenAI, "Prompt Engineering Guide," 2024. [Online]. Available: <https://platform.openai.com/docs/guides/prompt-engineering>
- [31] F. Cirillo, *The Pomodoro Technique*. New York, NY, USA: Currency, 2018.
- [32] L. Richardson and S. Ruby, *RESTful Web Services*. Sebastopol, CA, USA: O'Reilly Media, 2007.
- [33] M. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [34] Meta AI, "Llama 3 Model Card," Meta Platforms, 2024. [Online]. Available: <https://ai.meta.com/llama/>